บทที่ 4 แบบจำลองเชิงสัมพันธ์

Relational Model

วัตถุประสงค์

- สามารถอธิบายถึงวิวัฒนาการของแบบจำลองเชิงสัมพันธ์ได้
- สามารถอธิบายแนวคิดหลักของแบบจำลองเชิงสัมพันธ์ได้
- สามารถอธิบายการแทนข้อมูลในแบบจำลองเชิงสัมพันธ์ได้
- สามารถอธิบายการกำหนดเงื่อนไขบังคับสำหรับบูรณภาพของข้อมูลแบบต่างๆ ได้
- สามารถสร้างแบบจำลองเชิงสัมพันธ์จาก ER model ได้
- สามารถอธิบายความสำคัญของพจนานุกรมข้อมูล และสร้างพจนานุกรมข้อมูลได้

คำสำคัญ: กฎ 12 ข้อของ E. F. Codd (E. F. Codd's 12 rules); รีเลชัน (relation); สคีมา (schema); กรณีตัวอย่าง (instance); ทูเพิล (tuple); ฟิลด์ (field); เขต/โดเมน (domain); ดีกรี (degree); คาร์ดินัลลิตี (cardinality); เงื่อนไขบังคับบูรณภาพ (integrity constraints); เงื่อนไขบังคับโดเมน (domain constraints); เงื่อนไขบังคับคีย์ (key constraints); คีย์ (key); คีย์หลัก (primary key); คีย์คู่แข่ง (candidate key); คีย์ประกอบ (composite key); คีย์ภายนอก (foreign key); เงื่อนไขบังคับคีย์ภายนอก (foreign key constraints); บูรณภาพเชิงอ้างอิง (referential integrity); เงื่อนไขบังคับทั่วไป (general constraints); การสร้างแบบจำลองเชิงสัมพันธ์จาก ER model (ER model to relational model translation); วิว (view); พจนานุกรมข้อมูล (data dictionary)

บทที่ 4 แบบจำลองเชิงสัมพันธ์

Relational Model

4.1 บทนำ

แบบจำลองเชิงสัมพันธ์เป็นพื้นฐานของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ในขั้นตอนการออกแบบฐานข้อมูลเพื่อนำมาใช้ประโยชน์นั้น หลังจากการออกแบบระดับแนวคิดด้วย ER model แล้ว เราจำเป็นต้องเลือกแบบจำลองข้อมูลเพื่อแทนข้อมูลที่จะนำมาบริหารจัดการ ในปัจจุบันนั้นแบบจำลองเชิงข้อมูลเชิงสัมพันธ์ได้รับความนิยมสูงสุด ระบบจัดการฐานข้อมูลเชิงสัมพันธ์มีการนำมาใช้ในเชิงธุรกิจและในแวดวงอื่นๆ ซึ่งก่อให้เกิดประโยชน์อย่างมหาศาล ในบทนี้เราจะได้กล่าวถึงแบบจำลองข้อมูลเชิงสัมพันธ์และการออกแบบฐานข้อมูลระดับตรรกะโดยใช้แบบจำลองเชิงสัมพันธ์

แนวคิดทั่วไปของแบบจำลองเชิงสัมพันธ์ได้แก่วิวัฒนาการ และกฎที่ระบบฐานข้อมูลเชิงสัมพันธ์ต้องรองรับได้อธิบายในหัวข้อที่ 4.2.1 และ 4.2.2 ตามลำดับ ในหัวข้อที่ 4.2.3 อธิบายคำศัพท์พื้นฐานของแบบจำลองเชิงสัมพันธ์ หัวข้อที่ 4.3 อธิบายถึงการกำหนดเงื่อนไขบังคับต่างๆ ในแบบจำลองเชิงสัมพันธ์เพื่อบูรณภาพของข้อมูล จากนั้นในหัวข้อที่ 4.4 ได้อธิบายถึงวิวและความเกี่ยวข้องกับฐานข้อมูลระดับภายนอก

แบบจำลองเชิงสัมพันธ์นี้เป็นแบบจำลองข้อมูลระดับเชิงตรรกะของฐานข้อมูล เราสามารถออกแบบฐานข้อมูลโดยใช้แบบจำลองเชิงความหมาย (semantic model) ด้วย ER model ซึ่งไม่ยึดติดกับแบบจำลองข้อมูลใดๆ ER model ที่ได้มานี้หาจะนำมาพัฒนาเป็นระบบฐานข้อมูลจะต้องได้รับการแปลงให้เป็นฐานข้อมูลระดับตรรกะเสียก่อน ในหัวข้อ 4.5 เราได้อธิบายวิธีการในการแปลง ER model ให้อยู่ในรูปของแบบจำลองเชิงสัมพันธ์ สำหรับในหัวข้อที่ 4.6 เป็นการอธิบายเรื่องพจนานุกรมข้อมูลซึ่งเป็นเอกสารสำคัญในระบบจัดการฐานข้อมูลที่ใช้อธิบายความหมายของวัตถุต่างๆ ในฐานข้อมูล จากนั้นหัวข้อที่ 4.7 ได้ยกตัวอย่างกรณีศึกษาของการสร้างแบบจำลองเชิงสัมพันธ์จาก ER model ที่ได้ออกแบบไว้จากความต้องการของผู้ใช้ในบทที่แล้ว

โครงสร้างของบทอ้างอิงตาม และ ERD ของตัวอย่างที่เกี่ยวข้องกับตัวอย่าง “บริษัท” ดัดแปลงมาจาก Ramakrishnan and Gehrke, 2003 รายละเอียดอื่นๆ อ้างอิงจากและอ่านเพิ่มเติมได้ตามเอกสารอ้างอิง

4.2 แนวคิดทั่วไปเกี่ยวกับแบบจำลองเชิงสัมพันธ์

4.2.1 วิวัฒนาการของแบบจำลองเชิงสัมพันธ์

E. F. Codd เป็นผู้นำเสนอแบบจำลองข้อมูลเชิงสัมพันธ์เป็นคนแรกในปี ค.ศ. 1970 ในขณะนั้นระบบจัดการฐานข้อมูลส่วนใหญ่อยู่บนพื้นฐานของแบบจำลองข้อมูลเชิงลำดับชั้นและแบบจำลองข้อมูลเชิงเครือข่าย การเสนอรูปแบบของแบบจำลองเชิงสัมพันธ์นั้นนำมาซึ่งการปฏิรูปครั้งใหญ่ในวงการฐานข้อมูล รูปแบบของแบบจำลองข้อมูลที่ชัดเจน สอดคล้องกับการใช้งาน และมีทฤษฎีมารองรับนั้นก่อให้เกิดการศึกษา พัฒนาและใช้งานระบบจัดการฐานข้อมูลอย่างแพร่หลาย

ในทศวรรษที่ 1970 ห้องปฏิบัติการวิจัยของบริษัท IBM ในเมือง San José มลรัฐแคลิฟอร์เนีย ได้พัฒนาระบบจัดการฐานข้อมูลบนพื้นฐานของแบบจำลองข้อมูลเชิงสัมพันธ์ โดยให้ชื่อระบบว่า IBM System

R ซึ่งระบบดังกล่าวยังเป็นต้นแบบของการประยุกต์ระบบที่รองรับข้อกำหนดที่จำเป็นอื่นๆ ของระบบจัดการฐานข้อมูล เช่น การจัดการธุรกรรม การรองรับการทำงานพร้อมกัน การใช้งานข้อคำถามที่มีโครงสร้าง การปรับปรุงประสิทธิภาพของการแสดงข้อมูลจากข้อคำถาม ความมั่นคงปลอดภัยของข้อมูล บูรณภาพของข้อมูลหรือความถูกต้องของข้อมูล และส่วนติดต่อกับผู้ใช้ เป็นต้น System R นี้ยังเป็นโครงการที่เป็นต้นกำเนิดของภาษาที่ใช้ในการจัดการฐานข้อมูลเชิงสัมพันธ์ SQL ที่เป็นมาตรฐานในปัจจุบัน ตลอดจนเป็นต้นแบบของระบบจัดการฐานข้อมูลเชิงพาณิชย์ เช่น IBM DB/2 และ Oracle เป็นต้น

INGRES (Interactive Graphics Retrieval System) เป็นโครงการพัฒนาต้นแบบระบบจัดการฐานข้อมูลสำหรับฐานข้อมูลเชิงสัมพันธ์ที่พัฒนาโดย University of California at Berkeley โครงการดังกล่าวเกิดขึ้นในเวลาไล่เลี่ยกันกับ IBM System R ทั้งสองโครงการมีวัตถุประสงค์คล้ายคลึงกันในการพัฒนาระบบจัดการฐานข้อมูลเชิงสัมพันธ์และการเพิ่มความสามารถที่ทำให้ระบบทำงานได้อย่างมีประสิทธิภาพ จากความสำเร็จของต้นแบบของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ต่างๆ จึงมีการสร้างระบบฐานข้อมูลเชิงสัมพันธ์เชิงพาณิชย์ขึ้นในช่วงปลายทศวรรษที่ 1970 เป็นต้นมา และยังคงมีใช้แพร่หลายในปัจจุบัน ระบบจัดการฐานข้อมูลที่ได้รับความนิยมได้แก่ Microsoft Access, Microsoft SQL Server, Microsoft Visual FoxPro, Oracle, IBM DB/2, MySQL ฯลฯ

4.2.2 กฎ 12 ข้อของ E. F. Codd (E. F. Codd' 12 Rules)

กฎ 12 ข้อของ E. F. Codd คือ กฎที่บัญญัติไว้สำหรับกำหนดให้ระบบจัดการฐานข้อมูลเชิงสัมพันธ์รองรับ

ในปี ค.ศ. 1985 Edgar Frank Codd ผู้ให้กำเนิดแบบจำลองข้อมูลเชิงสัมพันธ์ได้วางกฎไว้สำหรับเป็นบรรทัดฐานของการทำงานของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ต่างๆ เพื่อให้เป็นระบบจัดการฐานข้อมูลที่รองรับแบบจำลองเชิงสัมพันธ์โดยสมบูรณ์ โดยกำหนดเป็นกฎ 12 ข้อ (E. F. Codd's 12 rules) ซึ่งโดยแท้จริงแล้วมี 13 ข้อหากรวมข้อที่ 0 ดังนี้

กฎข้อที่ 0 กฎพื้นฐาน (Foundation rule)

ระบบฐานข้อมูลเชิงสัมพันธ์ต้องมีความสามารถในจัดการฐานข้อมูลทั้งหมดโดยใช้ความสามารถเชิงสัมพันธ์ กล่าวคือการจัดการข้อมูลใดๆ จะอ้างอิงถึงทฤษฎีเชิงสัมพันธ์เท่านั้น

กฎข้อที่ 1 กฎสารสนเทศ (Information rule)

ข้อมูลทั้งหมดในฐานข้อมูลเชิงสัมพันธ์จะต้องปรากฏอยู่อย่างชัดแจ้ง เช่นข้อมูลในฐานข้อมูลแต่ละระเบียนที่จะต้องปรากฏอยู่ในตาราง ตลอดจนระบุด้วยชื่อของตาราง และชื่อของคอลัมน์ และต้องจัดเก็บโดยแทนค่าแบบใดแบบหนึ่งเท่านั้น

กฎข้อที่ 2 กฎการรับประกันการเข้าถึงข้อมูล (Guaranteed access rule)

ข้อมูลทุกๆ ค่าที่เก็บไว้ในฐานข้อมูลจะต้องสามารถเข้าถึงได้โดยการระบุชื่อตาราง คีย์หลัก และชื่อคอลัมน์

กฎข้อที่ 3 การรองรับค่าว่าง (Systematic null value support)

ระบบจัดการฐานข้อมูลต้องรองรับค่าว่างหรือ null value โดยค่าดังกล่าวต้องแสดงว่าข้อมูลยังคงว่างเปล่าอยู่ ต่างจากค่าโดยปริยาย เช่น ต่างจากค่า 0 และไม่ขึ้นกับโดเมนของแอททริบิวต์ใดๆ

กฎข้อที่ 4 โครงสร้างของรีเลชันสามารถเรียกดูได้ (Dynamic On-line Catalog Based on the Relational Model)

โครงสร้างต่างๆ ของฐานข้อมูลที่นิยามหรือกำหนดไว้เช่น รายการตารางต่างๆ ในฐานข้อมูล รายการคอลัมน์ในตาราง จะต้องสามารถเรียกแสดงโครงสร้างและจัดการแก้ไขได้โดยภาษาที่ใดๆ ที่มีโครงสร้าง ซึ่งโครงสร้างเหล่านี้จะถูกอ้างอิงโดยข้อมูลฐานข้อมูล ตลอดจนผู้ใช้งานที่ใช้งานข้อมูลจะอาศัยโครงสร้างเดียวกันนี้ในการใช้งานฐานข้อมูล

กฎข้อที่ 5 กฎการมีภาษาที่สนับสนุนอย่างเต็มรูปแบบ (Comprehensive data sublanguage rule)

จะต้องมีภาษาอย่างน้อยหนึ่งภาษาที่ออกแบบมาอย่างสมบูรณ์และสามารถจัดการฐานข้อมูลเชิงสัมพันธ์ได้เต็มความสามารถของแบบจำลอง (อาจไม่ใช่ SQL ก็ได้) โดยความสามารถที่ภาษาจะต้องรองรับได้แก่

1) การนิยามข้อมูล
2) การนิยามวิว
3) การแก้ไขข้อมูล ได้แก่การเพิ่ม ลบ และแก้ไข
4) การกำหนดกฎบูรณภาพหรือกฎเพื่อคงความถูกต้องของข้อมูล
5) การกำหนดสิทธิในการเข้าใช้ฐานข้อมูล
6) การจัดการธุรกรรม

กฎข้อที่ 6 กฎการแก้ไขข้อมูลผ่านทางวิว (View updating rule)

วิวทุกวิวที่อนุญาตให้แก้ไขข้อมูลได้ จะต้องสามารถแก้ไขข้อมูลในฐานข้อมูลผ่านทางวิวได้ กล่าวคือโดยปกติวิวจะเกิดจากรีเลชันหลักในฐานข้อมูล โอยอาจเป็นหลายๆ รีเลชันมารวมกัน การแก้ไขข้อมูลในวิวใดๆ ข้อมูลที่ได้รับการแก้ไข ข้อมูลในรีเลชันหลักที่สัมพันธ์กับข้อมูลในวิวจะต้องถูกปรับปรุงให้ตรงกัน

กฎข้อที่ 7 มีความสามารถในการเพิ่ม ลบ และแก้ไขข้อมูล (High-level insertion, update, and deletion)

ไม่ใช่แค่การแสดงข้อมูลเท่านั้นที่ระบบจัดการฐานข้อมูลรองรับ ระบบจัดการฐานข้อมูลต้องรองรับการเพิ่ม การแก้ไขหรือปรับปรุง และการลบข้อมูลได้ ซึ่งกฎข้อนี้กล่าวถึงความสามารถในการจัดการข้อมูลทีละหลายๆ กลุ่มของข้อมูลเช่น หลายๆ ระเบียน

กฎข้อที่ 8 มีความเป็นอิสระของข้อมูลระดับกายภาพ (Physical data independence)

โปรแกรมประยุกต์หรือโปรแกรมอื่นใดที่ใช้งานระบบจัดการฐานข้อมูลจะต้องไม่ได้รับผลกระทบกรณีมีการปรับเปลี่ยนข้อกำหนดด้านกายภาพ กล่าวคือโปรแกรมที่เรียกใช้งานระบบจัดการฐานข้อมูลจะต้อง

ยังคงทำงานได้ดังเดิมในกรณีที่โครงสร้างภายในของระบบจัดการฐานข้อมูลในระดับการจัดเก็บข้อมูลในอุปกรณ์หน่วยความจำสำรอง (ฮาร์ดดิสก์) เช่น อาจมีการเปลี่ยนลำดับของไฟล์ใหม่ มีการสร้างไฟล์ดัชนีใหม่ เป็นต้น

กฎข้อที่ 9 มีความเป็นอิสระของข้อมูลระดับตรรกะ (Logical data independence)

โปรแกรมประยุกต์หรือโปรแกรมอื่นใดที่ใช้งานระบบจัดการฐานข้อมูลจะต้องไม่ได้รับผลกระทบกรณีมีการปรับเปลี่ยนข้อกำหนดด้านตรรกะ กล่าวคือโปรแกรมที่เรียกใช้งานระบบจัดการฐานข้อมูลจะต้องยังคงทำงานได้ดังเดิมในกรณีที่มีการเปลี่ยนโครงสร้างตารางข้อมูล อย่างไรก็ตาม หากโครงสร้างข้อมูลที่ปรับเปลี่ยนเกี่ยวข้องกับโปรแกรมประยุกต์นั้นๆ โดยตรงเช่นจะต้องมีการจัดเก็บข้อมูลเพิ่มอีก 1 คอลัมน์ ผ่านทางหน้าจอโปรแกรม เรายังคงต้องแก้ไขโปรแกรมประยุกต์ ให้รองรับการกรอกข้อมูลนั้น

กฎข้อที่ 10 มีความเป็นอิสระของบูรณภาพ (Integrity independence)

ภาษาฐานข้อมูลต้องรองรับการกำหนดข้อกำหนดหรือกฎต่างๆ ที่บังคับให้ข้อมูลมีความถูกต้องตามข้อกำหนดทางธุรกิจหรือที่เรียกว่าบูรณภาพของข้อมูล ข้อกำหนดเหล่านี้ต้องถูกจัดเก็บอยู่ในระบบจัดการฐานข้อมูลและไม่สามารถที่จะละเมิดข้อกำหนดนี้ได้

กฎข้อที่ 11 มีอิสระในการกระจาย (Distribution independence)

โปรแกรมประยุกต์หรือโปรแกรมอื่นใดที่ใช้งานระบบจัดการฐานข้อมูลจะต้องไม่ได้รับผลกระทบสามารถใช้งานได้อย่างเป็นปกติ ในกรณีที่มีการกระจายของข้อมูล ซึ่งในที่นี้หมายถึงการที่ข้อมูลกระจายไปอยู่ในอุปกรณ์จัดเก็บหลายๆ แหล่ง หรือแม้แต่กลับมารวมกันในแหล่งเดียว

กฎข้อที่ 12 ไม่อนุญาตให้ภาษาในระดับต่ำกว่าเลี่ยงกฎบูรณภาพ (Nonsubversion rule)

ภาษาในระดับต่ำกว่า ในที่นี้หมายถึงภาษาที่จัดการข้อมูลครั้งละข้อมูล เช่น ครั้งละระเบียน ครั้งละคอลัมน์ในระเบียน จะต้องไม่สามารถเลี่ยงกฎต่างๆ ที่ตั้งไว้เพื่อบูรณภาพของข้อมูลได้

4.2.3 ข้อดีของระบบจัดการฐานข้อมูลเชิงสัมพันธ์

นอกจากระบบจัดการฐานข้อมูลเชิงสัมพันธ์จะมีข้อดีครอบคลุมข้อดีของระบบจัดการฐานข้อมูลทั่วไปที่กล่าวถึงในบทที่ 1 แล้วยังมีข้อดีอื่นๆ อีกดังนี้

1. ระบบฐานข้อเชิงสัมพันธ์เป็นกลุ่มของรีเลชันหรือตารางความสัมพันธ์ ที่ข้อมูลถูกจัดเก็บเป็นแถว (Row) และคอลัมน์ (Column) ซึ่งสอดคล้องกับรูปแบบการใช้งานข้อมูล และทำให้เห็นภาพและเข้าใจง่าย
2. ผู้ใช้ไม่จำเป็นต้องทราบถึงวิธีการจัดเก็บและการจัดการข้อมูลในฐานข้อมูลก็สามารถใช้งานได้อย่างมีประสิทธิภาพ
3. มี SQL เป็นภาษาหลักในการใช้งานฐานข้อมูลซึ่งมีรูปแบบที่ง่ายต่อการใช้งาน
4. สามารถเชื่อมโยงข้อมูลได้ง่าย
5. ข้อมูลมีความเป็นอิสระในระดับต่างๆ ตามคุณสมบัติความเป็นอิสระของระบบจัดการฐานข้อมูล

4.2.4 คำศัพท์ที่เกี่ยวข้องกับแบบจำลองเชิงสัมพันธ์

E. F. Codd ผู้เสนอแบบจำลองเชิงสัมพันธ์อาศัยทฤษฎีทางคณิตศาสตร์ในการนำเสนอแบบจำลองเชิงสัมพันธ์ ซึ่งรีเลชันนั้นเปรียบได้กับตารางข้อมูลนั่นเอง ในหัวข้อนี้อธิบายถึงคำศัพท์ต่างๆ ที่เกี่ยวข้องกับแบบจำลองเชิงสัมพันธ์

<u>รีเลชัน (Relation)</u>

รีเลชัน คือ ตารางข้อมูลที่ประกอบไปด้วยคอลัมน์/สดมภ์ (Columns) และแถว (Rows)

ในระบบจัดการฐานข้อมูลเชิงสัมพันธ์นั้น รีเลชันคือตารางข้อมูลนั่นเอง โดยแท้ที่จริงแล้วรีเลชันหมายถึงความสัมพันธ์ของกลุ่มข้อมูล ซึ่งในที่นี่หมายถึงแอททริบิวต์ ซึ่งเป็นกลุ่มข้อมูลที่อธิบายเอนทิตีหรือความสัมพันธ์ระหว่างเอนทิตี กลุ่มข้อมูลเหล่านี้มีความสัมพันธ์กันในกลุ่ม การนำแอททริบิวต์ที่เกี่ยวข้องมารวมกันตรงกับลักษณะการใช้งานในรูปแบบของตารางข้อมูล นอกจากรีเลชันซึ่งเป็นพื้นฐานหลักของระบบฐานข้อมูลเชิงสัมพันธ์แล้ว ยังมีส่วนประกอบอื่นๆ อีกดังต่อไปนี้

<u>แอททริบิวต์ (Attribute)</u>

แอททริบิวต์ คือ คอลัมน์ของข้อมูลในตาราง

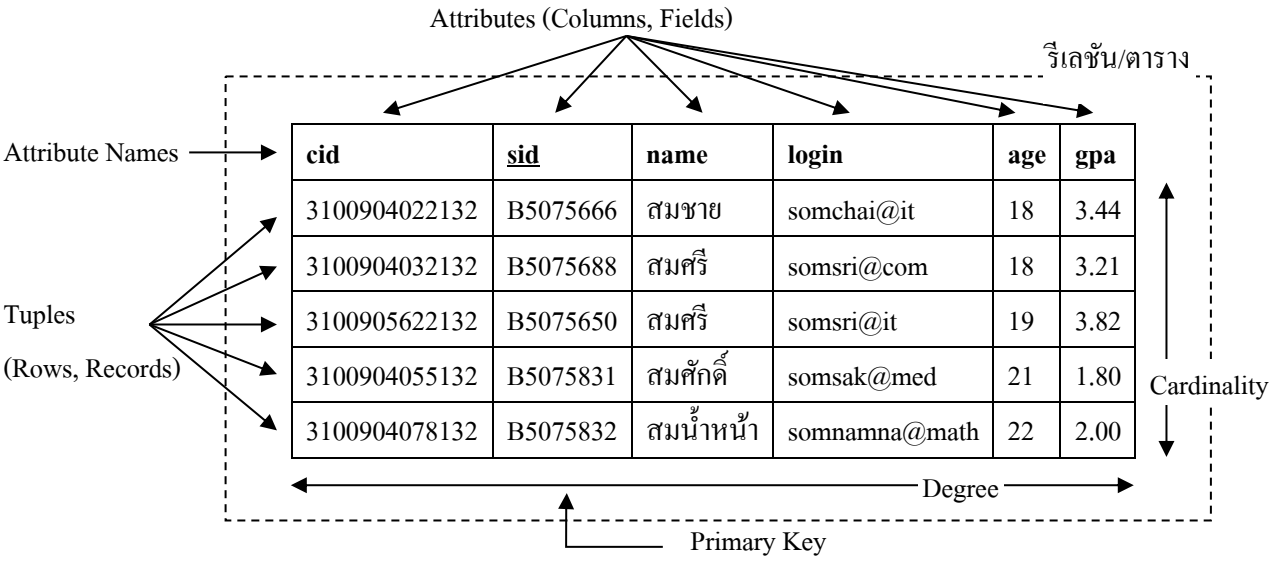
แอททริบิวต์ หรือ คอลัมน์ (column) หรือ ฟิลด์ (field) คือคุณสมบัติต่างๆ ที่อธิบายข้อมูลของเอนทิตีหรือความสัมพันธ์ระหว่างเอนทิตี สำหรับในระบบฐานข้อมูลเชิงสัมพันธ์ แอททริบิวต์ปรากฏอยู่ในรูปแบบของคอลัมน์ข้อมูลของตาราง เขตของค่าที่เป็นไปได้ของแอททริบิวต์เรียกว่า **โดเมน (domain)** การใช้งานฐานข้อมูลโดยปกติเรามีความจำเป็นต้องกำหนดเขตของข้อมูลในแต่ละแอททริบิวต์เพื่อให้ข้อมูลมีความถูกต้อง เช่น เพศของคนแต่ละคนมีได้ 2 ค่าคือ ชายและหญิง เราสามารถกำหนด**เงื่อนไขบังคับโดเมน (domain constraint)** ในระบบจัดการฐานข้อมูล เพื่อบังคับให้ข้อมูลที่บรรจุอยู่ในฐานข้อมูลมีความถูกต้องได้

<u>ทูเพิล (Tuple)</u>

ทูเพิล คือ แถวของข้อมูลในตาราง

ทูเพิล หรือ แถว (row) หรือ เรคอร์ด/ระเบียน (record) คือชุดของข้อมูล 1 ข้อมูลของรีเลชันนั้นๆ มีลักษณะเป็นข้อมูลที่อยู่ในแถวๆ หนึ่ง ซึ่งได้แก่ข้อมูลของเอนทิตีหรือความสัมพันธ์ระหว่างเอนทิตีนั่นเอง กลุ่มของแอททริบิวต์ทั้งหมดเป็นแม่แบบของข้อมูลแต่ละระเบียน เราจึงเรียกกลุ่มของแอททริบิวต์ที่เป็นตัวกำหนดโครงสร้างของรีเลชันว่า **สคีมา (schema)** และสำหรับข้อมูลที่เป็นไปตามโครงสร้างของสคีมาในรีเลชันเรียกว่า **กรณีตัวอย่าง (instance)** ของสคีมานั้นๆ ดังเช่นตัวอย่างรีเลชันของนักศึกษา (Student) ที่แสดงในภาพที่ 4-1 ในหน้าถัดไป ตัวอย่างของรีเลชันนักศึกษาประกอบไปด้วยแอททริบิวต์หรือคอลัมน์ 5 คอลัมน์ ได้แก่ sid, name, login, age และ gpa ซึ่งทั้ง 5 คอลัมน์นี้สามารถเขียนให้อยู่ในรูปเค้าร่างหรือสคีมาของรีเลชันดังนี้

Student (*id:* string, *<u>sid</u>:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)



cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

ภาพที่ 4-1 รีเลชันหรือตารางความสัมพันธ์นักศึกษา

แอททริบิวต์แต่ละแอททริบิวต์ในสคีมาถูกกำหนดไว้ด้วยโดเมน เช่น sid มีโดเมนเป็น string คือสายอักขระ และ age เป็น integer หมายถึงต้องเป็นจำนวนเต็ม เป็นต้น กรณีตัวอย่าง (instance) ของสคีมานักศึกษาคือข้อมูลในตาราง ทูเพิลของรีเลชันคือแถวแต่ละแถวในตาราง นอกจากนี้ยังมีศัพท์เฉพาะเพิ่มเติมสำหรับรีเลชันดังนี้

<u>คีย์หลัก (Primary Key)</u>

คีย์หลัก คือ แอททริบิวต์หรือกลุ่มของแอททริบิวต์ที่สามารถระบุทูเพิลได้อย่างเฉพาะเจาะจง

ข้อมูลแต่ละระเบียนจะต้องสามารถเข้าถึงได้อย่างเฉพาะเจาะจงจากการระบุค่าแอททริบิวต์ใดแอททริบิวต์หนึ่ง หรือกลุ่มแอททริบิวต์ก็ได้ จากตัวอย่างเรากำหดให้ sid ซึ่งเป็นรหัสนักศึกษาเป็นคีย์หลัก ในสคีมาเราใช้การ<u>ขีดเส้นใต้</u>เป็นสัญลักษณ์ของคีย์หลัก อย่างไรก็ตามเราอาจใช้ cid ซึ่งเป็นรหัสประจำตัวประชาชนเป็นคีย์หลักก็ได้ โดยรายละเอียดเกี่ยวกับคีย์ได้อธิบายไว้ในหัวข้อ 4.3.2

<u>ดีกรี (Degree)</u>

ดีกรี คือ จำนวนแอททริบิวต์ในรีเลชัน

จากตัวอย่างรีเลชันนักศึกษามีค่าของดีกรีเป็น 6

คาร์ดินัลลิตี (Cardinality)

คาร์ดินัลลิตี คือ จำนวนของแถวในรีเลชัน

จากตัวอย่างรีเลชันนักศึกษามีคาร์ดินัลลิตีเท่ากับ 5

สรุปศัพท์ที่เกี่ยวข้องกับแบบจำลองเชิงสัมพันธ์ได้ดังนี้

ศัพท์เฉพาะ	**ศัพท์ทั่วไป**
รีเลชัน (relation)	ตาราง (table)
ทูเพิล (tuple)	แถว (row) หรือ เรคอร์ด/ระเบียน (record)
แอททริบิวต์ (attribute)	คอลัมน์/สดมภ์ (column) หรือ ฟิลด์ (field)
คาร์ดินัลลิตี (cardinality)	จำนวนแถว (number of rows)
ดีกรี (degree)	จำนวนแอททริบิวต์ (number of attributes)
คีย์หลัก (primary key)	ค่าเอกลักษณ์ (unique identifier)
โดเมน (domain)	เขตของข้อมูล (pool of legal values)

ฐานข้อมูลเชิงสัมพันธ์ (relational database) คือกลุ่มของรีเลชัน สคีมาของฐานข้อมูลเชิงสัมพันธ์ (relational database schema) คือกลุ่มของสคีมาของรีเลชันต่างๆ ในฐานข้อมูล นอกจากสคีมาที่กำหนดโครงสร้างและโดเมนของข้อมูลแล้ว การทำให้ข้อมูลมีความถูกต้องตรงตามข้อกำหนดทางธุรกิจยังสามารถกำหนดไว้ได้ในสคีมาของฐานข้อมูลเชิงสัมพันธ์ ดังจะได้กล่าวถึงในหัวข้อถัดไป

4.3 เงื่อนไขบังคับบูรณภาพ (Integrity Constraints)

เงื่อนไขบังคับบูรณภาพ คือ เงื่อนไขที่กำหนดไว้ในฐานข้อมูลเพื่อให้ข้อมูลในฐานข้อมูลมีความถูกต้องตรงตามลักษณะการใช้งาน

ฐานข้อมูลคือตัวข้อมูลที่เก็บไว้ในอุปกรณ์จัดเก็บเท่านั้น การเรียกใช้ และจัดการข้อมูลจำเป็นต้องอาศัยระบบจัดการฐานข้อมูลในการเข้าถึงข้อมูล การจะกำหนดเงื่อนไขให้ข้อมูลที่จะบรรจุลงในฐานข้อมูลมีความถูกต้องเราจำเป็นต้องอาศัยระบบจัดการฐานข้อมูลเช่นเดียวกัน การกำหนดเงื่อนไขบังคับบูรณภาพ (integrity constraint) นั้นเป็นการกำหนดเงื่อนไขบางประการให้ข้อมูลที่จะบรรจุอยู่ในฐานข้อมูลมีความถูกต้องตรงตามข้อกำหนดการใช้งาน การกำหนดเงื่อนไขบังคับต่างๆ สามารถกระทำได้โดยการกำหนดเงื่อนไขบังคับไว้ในสคีมาของฐานข้อมูล ระบบจัดการฐานข้อมูลจะอาศัยเงื่อนไขบังคับที่ระบุไว้ในฐานข้อมูลนี้ในการตรวจสอบข้อมูลต่างๆ ก่อนทำการบรรจุข้อมูลลงในฐานข้อมูลว่าตรงตามเงื่อนไขหรือไม่ หากไม่ตรงจะไม่สามารถบรรจุข้อมูลลงไปในฐานข้อมูลได้และจำต้องมีการจัดการกับความผิดพลาดที่เกิดขึ้นนี้ต่อไป

เงื่อนไขบังคับเพื่อทำให้ข้อมูลมีบูรณภาพที่สามารถกำหนดได้ในฐานข้อมูลเชิงสัมพันธ์มีหลายชนิด เงื่อนไขบังคับโดเมน และเงื่อนไขบังคับคีย์เป็นตัวอย่างของเงื่อนไขบังคับบูรณภาพที่ได้กล่าวถึงไปแล้ว หัวข้อต่อไปนี้จะอธิบายถึงเงื่อนไขบังคับอื่นๆ ที่มีความจำเป็นโดยทั่วไปในการใช้งานระบบจัดการฐานข้อมูลเชิงสัมพันธ์

4.3.1 เงื่อนไขบังคับคีย์ (Key Constraints)

เงื่อนไขบังคับคีย์ คือ การกำหนดให้แอททริบิวต์หรือกลุ่มของแอททริบิวต์ใดเป็นคีย์ของรีเลชันสำหรับการระบุระเบียนใดระเบียนหนึ่งได้อย่างเฉพาะเจาะจง

คีย์คือค่าของแอททริบิวต์ หรือชุดของแอททริบิวต์ที่มีจำนวนน้อยที่สุดที่สามารถระบุข้อมูลระเบียนใดระเบียนหนึ่งได้ ยกตัวอย่างเช่น ในรีเลชันนักศึกษา รหัสนักศึกษาสามารถระบุระเบียนของนักศึกษาคนใดคนหนึ่งได้ ในขณะที่เราไม่สามารถใช้แอททริบิวต์ชื่อของนักศึกษา หรืออายุเป็นคีย์ได้เนื่องจากมีระเบียนที่ชื่อของนักศึกษาซ้ำกัน การระบุเพียงชื่อ “สมศรี” ไม่สามารถระบุได้ว่าเป็นสมศรีที่มีรหัสนักศึกษา “B5075688” หรือ “B5075650” ดังนั้นข้อมูลในแอททริบิวต์ดังกล่าวจึงห้ามซ้ำกัน เราสามารถกำหนดเงื่อนไขบังคับเพื่อไม่ให้แอททริบิวต์ใด หรือแอททริบิวต์กลุ่มใดมีค่าไม่ซ้ำกัน และประกาศว่าแอททริบิวต์ดังกล่าวเป็นคีย์หลักของรีเลชันได้โดยการกำหนดเงื่อนไขบังคับคีย์ (key constraint) ไว้ในสคีมาของฐานข้อมูล คีย์สามารถแบ่งออกได้เป็นหลายประเภทตามลักษณะต่างๆ ของคีย์ดังนี้

1) **Simple key** หรือ **คีย์อย่างง่าย** หมายถึงคีย์ที่ประกอบด้วยแอททริบิวต์เดียว จากตัวอย่างรีเลชันนักศึกษา รหัสนักศึกษาเป็นคีย์ของรีเลชันที่เป็นแอททริบิวต์เดียวเท่านั้น โดยทั่วไปแล้วคีย์หลักของตารางมีลักษณะเป็น simple key
2) **Composite key (compound key หรือ concatenated key)** หรือ **คีย์ประกอบ** หมายถึงคีย์ที่ประกอบด้วย แอททริบิวต์ มากกว่า 1 แอททริบิวต์ เรากล่าวว่านอกจากคีย์จะหมายถึงแอททริบิวต์หนึ่งแอททริบิวต์ที่สามารถระบุระเบียนได้อย่างเฉพาะเจาะจงแล้ว คีย์ยังสามารถเป็นชุดของแอททริบิวต์ได้ด้วย โดยเป็นชุดที่มีจำนวนแอททริบิวต์ที่น้อยที่สุดที่สามารถระบุระเบียนได้ เช่น ในกรณีตัวอย่างข้อมูลของรีเลชันนักศึกษา เราอาจกำหนดให้ชื่อและอายุ {*name, age*} เป็นชุดหรือเซตของแอททริบิวต์ที่ใช้เป็นคีย์สำหรับรีเลชันได้ ในกรณีนี้ {*name, age*} เป็นคีย์ชนิดคีย์ประกอบ พิจารณาข้อมูลในตารางนักศึกษาพบว่า ถึงแม้จะมีนักศึกษาที่ชื่อ (*name*) สมศรี อยู่ถึง 2 คน แต่หากเราระบุอายุไปด้วย เราจะสามารถระบุได้ว่าเป็นสมศรีคนใดเนื่องจากสมศรีคนหนึ่งมีอายุ 18 และอีกคนหนึ่งมีอายุ 19
3) **Candidate key** หรือ **คีย์คู่แข่ง**หมายถึงคีย์ที่สามารถจะเป็นคีย์หลักของของรีเลชันได้ เช่น ในรีเลชันนักศึกษา เราอาจเลือกใช้รหัสประจำตัวประชาชน (*cid*) เป็นคีย์หลักของรีเลชันได้เนื่องจากรหัสประจำตัวประชาชนนั้นไม่ซ้ำกัน นอกจากนี้แอททริบิวต์หรือเซตของแอททริบิวต์ที่สามารถเป็นคีย์ของรีเลชันอาจได้แก่ {*login*} หรือ {*name, age*} หรือ {gpa} ฯลฯ ซึ่งเป็นคีย์ที่สามารถระบุระเบียนใดๆ ได้อย่างเฉพาะเจาะจง
4) **Primary key** หรือ **คีย์หลัก** หมายถึงคีย์คู่แข่งตัวหนึ่งที่ถูกเลือกขึ้นมาเป็นคีย์หลัก เราจะใช้สัญลักษณ์<u>ขีดเส้นใต้</u>กำกับไว้ใต้แอททริบิวต์ตัวนั้น ในการเขียนสคีมาของรีเลชัน จากตัวอย่างของคีย์คู่แข่ง เซตของแอททริบิวต์ที่สามารถเป็นคีย์หลักได้แก่ {*sid*}, {*cid*}, {*login*}, {*name, age*} หรือ {gpa} ฯลฯ อย่างไรก็ตามคีย์หลักนั้นควรเป็นคีย์ที่สามารถใช้งานได้อย่างสมเหตุสมผลและเหมาะสม พิจารณาคีย์ {*name, age*} นั้น ถึงแม้จะเป็นคีย์ได้ แต่ก็เป็นได้

เฉพาะในกรณีตัวอย่างของข้อมูลที่แสดง บ่อยครั้งที่นักศึกษาที่มีชื่อเหมือนกันจะมีอายุเท่ากัน หากเป็นเช่นนี้เราจะไม่สามารถใช้เซตแอททริบิวต์ {*name, age*} เป็นคีย์ได้อีกต่อไป เช่นเดียวกันกับ {*gpa*} ที่สามารถซ้ำกันได้ เราไม่นิยมนำข้อมูลที่อาจเปลี่ยนแปลงได้ง่ายมาใช้เป็นคีย์หลัก เช่น {*login*} สามารถเปลี่ยนแปลงได้ในกรณีที่นักศึกษาย้ายสาขาหรือเปลี่ยนชื่อ หรือแม้แต่เปลี่ยน login สำหรับรีเลชันนักศึกษานั้น คีย์ที่เหมาะสมสำหรับที่จะเป็นคีย์หลักได้แก่รหัสนักศึกษา {*sid*} รหัสประจำตัวประชาชนมีความยาวเกินไปในที่นี้ในการนำมาเป็นคีย์หลัก

5) **Secondary key (alternate key)** หรือ **คีย์รอง** หมายถึงคีย์คู่แข่ง ที่ไม่ได้ถูกเลือกให้เป็นคีย์หลัก
6) **Superkey** คือเซตของแอททริบิวต์ที่มีคีย์เป็นสมาชิก เช่น {*sid, name*} เซตของแอททริบิวต์ดังกล่าวสามารถระบุระเบียนได้อย่างเฉพาะเจาะจงเนื่องจากมีคีย์ sid อยู่แล้ว อย่างไรก็ตาม name เป็นแอททริบิวต์ที่ไม่มีความจำเป็นในการกำหนดเป็นคีย์ เราไม่ค่อยได้ใช้ประโยชน์ของ Superkey
7) **Foreign key** หรือ **คีย์ภายนอก** หมายถึงแอททริบิวต์ในรีเลชันหนึ่งที่อ้างอิงมาจากคีย์หลักของรีเลชันอื่น ซึ่งจะได้อธิบายในหัวข้อถัดไป

4.3.2 เงื่อนไขบังคับคีย์ภายนอก (Foreign Key Constraints)

เงื่อนไขบังคับคีย์บังคับภายนอก คือ ข้อกำหนดของข้อมูลที่อ้างอิงกันระหว่างรีเลชันเพื่อให้ข้อมูลมีความสอดคล้องกัน

ข้อมูลที่บรรจุอยู่ในรีเลชันหนึ่งมักอ้างถึงข้อมูลที่บรรจุอยู่ในอีกรีเลชันหนึ่ง หากข้อมูลที่บรรจุอยู่ในรีเลชันหนึ่ง หากข้อมูลนั้นๆ ในรีเลชันหนึ่งจะได้รับการแก้ไข เรามีความจำเป็นต้องตรวจสอบข้อมูลในอีกรีเลชันหนึ่งที่อ้างอิงถึงกันซึ่งต้องได้รับการแก้ไขหรือดำเนินการอย่างหนึ่งอย่างใดให้ข้อมูลที่อ้างถึงกันนี้มีความสอดคล้องกัน เงื่อนไขบังคับตรวจสอบความสอดคล้องกันของรีเลชัน 2 รีเลชันที่ใช้กันโดยทั่วไปได้แก่ **เงื่อนไขบังคับภายนอก** เช่น การกำหนดให้พนักงานทำงานประจำในแผนกใดๆ เราต้องสร้างความสัมพันธ์ระหว่างรหัสพนักงานกับรหัสของแผนก ซึ่งในบางครั้งผู้ป้อนข้อมูลอาจป้อนข้อมูลรหัสพนักงานผิดพลาดโดยไม่ปรากฏว่ามีรหัสพนักงานตามที่ป้อนในบริษัท เราจึงมีความจำเป็นต้องตรวจสอบก่อนว่าการกำหนดความสัมพันธ์ระหว่างพนักงานกับแผนกโดยรหัสพนักงานและรหัสแผนกนั้น รหัสพนักงานที่ป้อนเข้าไปในฐานข้อมูลจำเป็นต้องมีอยู่ในตารางข้อมูลพนักงานแล้ว เช่นเดียวกันกับรหัสแผนกที่จะต้องมีอยู่ในตารางของแผนก เราสามารถยกตัวอย่างอื่นๆ ประกอบกรณีตัวอย่างข้อมูลให้เห็นภาพได้ชัดเจนขึ้นดังนี้

จากสคีมาระบบฐานข้อมูลของมหาวิทยาลัยในบทที่ 1 แสดงรีเลชันการลงทะเบียนเรียนของนักศึกษาได้ดังนี้

Enrolled (*sid:* string, *cid:* string, *grade:* string)

ในที่นี้ Enrolled คือรีเลชันการลงทะเบียน sid คือแอททริบิวต์รหัสนักศึกษา cid และ grade คือผลการเรียนในรายวิชานั้น ข้อมูลแต่ละระเบียนที่เกิดจากสคีมานี้หมายถึงนักศึกษาคนใด ลงทะเบียนเรียนรายวิชา

อะไรไปแล้ว และได้เกรดเท่าใด เพื่อให้แน่ใจว่านักศึกษาที่ลงทะเบียนเรียนในรายวิชาต่างๆ มีอยู่จริง (เพื่อป้องกันข้อมูลผิดพลาดอันอาจจะเกิดจากการกรอกข้อมูล) รหัสนักศึกษาที่จะเพิ่มในรีเลชัน Enrolled ต้องปรากฏอยู่ในรีเลชันนักศึกษา เราเรียกแอททริบิวต์ sid ในตาราง Enrolled นี้ว่าเป็นคีย์ภายนอก (foreign key) ที่อ้างถึง (refer) รหัสนักศึกษา sid ในตารางนักศึกษา (Student) แอททริบิวต์ทั้งสองต้องมีชนิดของข้อมูลที่สอดคล้องกัน แต่ไม่จำเป็นต้องใช้ชื่อเดียวกัน แสดงได้ดังกรณีตัวอย่างข้อมูลต่อไปนี้

Foreign Key

cid	*grade*	*sid*
204203	A	B5075650
204204	B	B5075831
204205	D	B5075688
204329	F	B5075666

Enrolled (Referencing relation)

Primary Key

cid	*sid*	*name*	*login*	*age*	*gpa*
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

Student (Referenced relation)

ภาพที่ 4-2 บูรณภาพเชิงอ้างอิง (Referential Integrity)

จากตัวอย่าง รหัสนักศึกษาทั้งหมดในรีเลชัน Enrolled ต้องอ้างอิงมาจากรีเลชัน Student แต่รหัสนักศึกษาไม่จำเป็นต้องอ้างอิงมาจากรีเลชัน Enrolled มีนักศึกษาบางคนที่ยังไม่ได้ลงทะเบียนเรียน เช่น “สมน้ำหน้า” ที่มีรหัสนักศึกษาคือ “B5075832” และไม่ปรากฏในรีเลชัน Enrolled

ถ้าเราทำการเพิ่มระเบียนใหม่ในรีเลชัน Enrolled ด้วยข้อมูล <“B5075833”, 204203, C> การกำหนดเงื่อนไขบังคับคีย์ภายนอก ทำให้เราไม่สามารถเพิ่มข้อมูลนี้ได้เนื่องจากไม่มีนักศึกษารหัส B5075833 ปรากฏอยู่ รหัสนักศึกษานี้คล้ายคลึงกับ B5075832 ซึ่งอาจเป็นการพิมพ์ข้อมูลที่ผิดพลาด ทำให้ผู้ใช้สามารถจัดการแก้ไขข้อมูลให้ถูกต้องได้ นอกจากนี้หากมีการลบนักระเบียนนักศึกษา B5075831 ออก ระบบจัดการฐานข้อมูลจะไม่อนุญาตให้ดำเนินการเนื่องจากมีการอ้างถึงนักศึกษาดังกล่าวจากตาราง Enrolled ซึ่งลงทะเบียนเรียนไว้ในรายวิชา 204204 อย่างไรก็ตามเราอาจกำหนดให้ระบบจัดการฐานข้อมูลดำเนินการอย่างหนึ่งอย่างใดนอกเหนือจากการไม่อนุญาตให้ระเบียนนักศึกษาไม่สามารถลบได้ เช่น อาจกำหนดให้ทำการลบข้อมูลทุกระเบียนที่อ้างถึงรหัสนักศึกษาคนนั้นออกจากตาราง Enrolled ด้วยก็ได้ ซึ่งขึ้นอยู่กับข้อกำหนดและลักษณะการใช้งานจริง

เงื่อนไขบังคับภายนอกนี้ทำให้ข้อมูลมีความถูกต้องในการอ้างอิงข้อมูลซึ่งเราเรียกลักษณะความมีบูรณภาพของข้อมูลในลักษณะนี้ว่า**บูรณภาพเชิงอ้างอิง (referential integrity)** และการกำหนดเงื่อนไขบังคับนี้เป็นการ**บังคับให้ข้อมูลมีบูรณภาพเชิงอ้างอิง (enforce referential integrity)** แม้ว่าเงื่อนไขบังคับคีย์ภายนอกนี้จะใช้ชื่อดังกล่าวแต่เราสามารถอ้างอิงแอททริบิวต์ภายในรีเลชันเดียวกันก็ได้

4.3.3 เงื่อนไขบังคับทั่วไป (General Constraints)

เงื่อนไขบังคับทั่วไป คือ ข้อกำหนดของข้อมูลอื่นๆ ซึ่งอาจเกี่ยวข้องกับความหมายของข้อมูลหรือข้อกำหนดทางธุรกิจ

เงื่อนไขบังคับโดเมน คีย์ และคีย์ภายนอก เป็นเงื่อนไขบังคับโดยทั่วไปที่ระบบจัดการฐานข้อมูลรองรับ นอกจากนี้ระบบจัดการฐานข้อมูลยังมีความสามารถในการรองรับเงื่อนไขบังคับอื่นๆ อีกที่นอกเหนือจากเงื่อนไขบังคับที่ได้กล่าวถึง แต่มีความจำเป็นในการกำหนดเพื่อให้รองรับกับการใช้งานจริงหรือข้อกำหนดทางธุรกิจ เช่น นักศึกษาที่มีอายุตั้งแต่ 18 ปีขึ้นไป ต้องมีเกรดเฉลี่ยมากกว่า 2.75 เป็นต้น หากมีการเพิ่มหรือแก้ไขข้อมูลนักศึกษาที่ไม่ตรงตามเงื่อนไขจะไม่สามารถกระทำได้และจะต้องมีการจัดการความผิดพลาดนี้ นอกจากนี้ยังรวมถึงเงื่อนไขบังคับที่ไม่ซับซ้อนนักเช่นคอลัมน์นั้นๆ จะต้องปรากฏค่าอยู่ในกรณีที่มีการแทรกข้อมูลระเบียนใหม่ เป็นต้น

4.4 วิว (View)

วิว คือ มุมมองข้อมูลของผู้ใช้ซึ่งอยู่ในรูปแบบของตารางที่อาจเกิดจากรีเลชันหลายๆ รีเลชัน

ตารางข้อมูลในมุมมองของผู้ใช้นั้นอาจเป็นข้อมูลทีเกิดจากรีเลชันหลายๆ รีเลชันก็ได้ เราเรียกตารางข้อมูลตามมุมมองของผู้ใช้ว่าวิว ซึ่งวิวนี้ระบบจัดการฐานข้อมูลเชิงสัมพันธ์จะจัดเก็บไว้เพียงสคีมาของวิว กรณีตัวอย่างข้อมูลจริงจะเกิดจากการนำของมูลจากรีเลชันที่เกี่ยวข้องมาเชื่อมโยงกันและแสดงผลที่กำหนดโดยสคีมา

สคีมาของฐานข้อมูลระบบลงทะเบียนในมหาวิทยาลัย (Ramakrishnan and Gehrke, 2003) จากบทที่ 1 ประกอบไปด้วยรีเลชันดังนี้

Student(*cid:* string, *sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)

Faculty(*fid:* string, *fname:* string, *sal:* real)

Courses(*cid:* string, *cname:* string, *credits:* integer)

Rooms(*rno:* integer, *address:* string, *capacity:* integer)

Enrolled(*sid:* string, *cid:* string, *grade:* string)

Teaches(*fid:* string, *cid:* string)

Meets_In(*cid:* string, *rno*: integer, *time:* string)

พิจารณาตัวอย่างรีเลชัน Course_info ซึ่งเป็นวิวที่สร้างขึ้นเพื่อแสดงข้อมูลรายวิชา ประกอบด้วยรหัสวิชา cid ชื่ออาจารย์ผู้สอน fname และจำนวนผู้ลงทะเบียน enrollment

Course_info(*cid:* string, *fname:* string, *enrollment:* integer)

รีเลชัน Course_info นี้ไม่ได้มีตัวข้อมูลจริงเก็บไว้ในฐานข้อมูลแต่มีสร้างขึ้นเมื่อผู้ใช้เรียกดูข้อมูลผ่านทางวิวนี้ ซึ่งตารางข้อมูลที่ถูกสร้างขึ้นจะสร้างมาจากข้อมูลในตารางอื่น ได้แก่ cid จากตารางรายวิชา (Courses) fname ได้มาจากตารางการสอน (Teaches) รวมกับตารางอาจารย์ (Faculty) และ enrollment ได้มา

จากการคำนวณโดยนับจำนวนระเบียนที่นักศึกษาลงทะเบียนรายวิชานั้นๆ จากตารางลงทะเบียน (Enrolled) กรณีตัวอย่างของรีเลชัน Course_info และข้อมูลในตารางที่เกี่ยวข้องแสดงได้ดังนี้

ข้อมูลในตาราง

Enrolled

cid	*grade*	*sid*
204203	A	B5075650
204204	B	B5075831
204205	D	B5075688
204329	F	B5075666
204204	A	B5075688

Student

cid	*sid*	*name*	*login*	*age*	*gpa*
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00
3100904032233	B5075899	สมแล้ว	somlaew@math	22	2.00

Teaches

fid	*cid*
146009	204203
146009	204204
146009	204205
146009	204329
147008	204205

Faculty

fid	*fname*	*salary*
146009	สถิตย์	50000
147008	พอลล่า	50000

ข้อมูลที่เกิดขึ้นจากวิว Course_info โดยระบุรหัสวิชาเป็น 204204

Course_info

cid	*fname*	*enrollment*
204204	สถิตย์	2

โดย

cid: จากตาราง Courses

fname: จากตาราง Faculty และ Teaches ในกรณีนี้ในตาราง Teaches อาจารย์ผู้สอนมีรหัสประจำตัวอาจารย์คือ 146009 ซึ่งเมื่ออ้างอิงถึงชื่ออาจารย์ในตาราง Faculty ได้แก่อาจารย์สถิตย์

enrollment: ได้จากการคำนวณโดยการนับจำนวนผู้เรียนรายวิชา 204204 ในตาราง Enrolled ซึ่งมีจำนวน 2 คนได้แก่นักศึกษารหัส B5075831 และ B5075688

การสร้างวิวเป็นการสร้างมุมมองของผู้ใช้ซึ่งเป็นการรองรับฐานข้อมูลในระดับภายนอก ทำให้เกิดประโยชน์ดังนี้

- ความเป็นอิสระของข้อมูล กล่าวคือหากมีการเปลี่ยนแปลงโครงสร้างตารางใดๆ เช่นมีการเพิ่มแอททริบิวต์ตำแหน่งทางวิชาการของตารางอาจารย์แล้ว วิว Course_info ไม่ได้รับผลกระทบใดๆ
- ความสะดวกจากการเรียกดูและจัดการข้อมูลที่ซับซ้อน จากตัวอย่างพบว่าข้อมูลที่กว่าจะได้มาเป็นตาราง Course_info นี้ต้องทำการเชื่อมโยงตารางหลายๆ ตารางเข้าด้วยกัน การสร้างวิวและเรียกใช้งานผ่านทางวิวจะทำให้เรากำหนดการเชื่อมโยงต่างๆ เพียงครั้งเดียว หลังจากนั้นสามารถเรียกดูและจัดการข้อมูลผ่านทางวิวซึ่งเปรียบเสมือนตารางเพียงตารางเดียวได้โดยสะดวก
- ขจัดปัญหาความซ้ำซ้อนและผิดพลาดของข้อมูล เช่นค่าของคอลัมน์ enrollment ในวิว Course_info เกิดจากการคำนวณและเราไม่ต้องจัดเก็บข้อมูลนี้ไว้เพราะจำนวนผู้ลงทะเบียนสามารถเปลี่ยนแปลงได้ตลอดเวลา
- เพิ่มความปลอดภัยของข้อมูล โดยการซ่อนข้อมูลจริงไว้เบื้องหลังวิว จากตัวอย่าง DBA สามารถกำหนดไม่ให้สิทธิผู้ใช้ทั่วไปเข้าถึงข้อมูลเงินเดือนของอาจารย์ได้ สามารถเรียกดูข้อมูลผ่านทางวิวเท่านั้น

ซึ่งทั้งหมดที่กล่าวมานี้เป็นเพียงข้อดีส่วนหนึ่งของวิวเท่านั้น

รีเลชันที่กำหนดขึ้นโดยไม่ได้มีข้อมูลบรรจุอยู่ในตัวรีเลชันเราเรียกว่า**วิว (View)** ในขณะที่รีเลชันที่บรรจุข้อมูลจริงในฐานข้อมูลซึ่งได้แก่รีเลชัน Student, Faculty, Teaches, Enrolled ในตัวอย่างนี้เราเรียกว่า**รีเลชันฐาน (base relation)** การสร้างวิวด้วย SQL อธิบายไว้ในบทที่ 7

4.5 การแปลง ER Model ให้เป็นแบบจำลองข้อมูลเชิงสัมพันธ์

หลังจากได้ทราบถึงแนวคิดของแบบจำลองเชิงสัมพันธ์และฐานข้อมูลเชิงสัมพันธ์แล้ว ในหัวข้อต่อไปนี้เป็นการอธิบายวิธีการแปลงแบบจำลองฐานข้อมูลที่ได้ออกแบบไว้ในระดับแนวคิดให้เป็นฐานข้อมูลในระดับตรรกะ ก่อนที่จะออกแบบฐานข้อมูลในระดับกายภาพในขั้นสุดท้ายต่อไป ER model ซึ่งเป็นแบบจำลองข้อมูลในระดับแนวคิดนั้นเป็นการออกแบบในระดับสูง ไม่ยึดติดกับระบบจัดการฐานข้อมูลรูปแบบใด แต่การจะจัดการฐานข้อมูลให้มีประสิทธิภาพเราอาศัยระบบจัดการฐานข้อมูลเชิงสัมพันธ์ในที่นี้เราต้องทำการแปลง ER model ให้อยู่ในรูปของแบบจำลองเชิงสัมพันธ์เพื่อนำไปใช้งานในระบบจัดการฐานข้อมูลเชิงสัมพันธ์ การแปลงให้อยู่ในรูปของฐานข้อมูลเชิงสัมพันธ์นั้นเราอาศัย SQL ในส่วนของการนิยามข้อมูล ใช้นิยามโครงสร้างของรีเลชันและกำหนดเงื่อนไขบังคับต่างๆ อย่างไรก็ดีแบบจำลองเชิงสัมพันธ์นั้นสามารถแทนได้ด้วยแผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ (relational database model diagram) ซึ่งใช้ในการอธิบายการแปลงจาก ER model ให้เป็นแบบจำลองเชิงสัมพันธ์

แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์นี้มีความคล้ายคลึงกับ ER model แต่มีการกำหนดรายละเอียดอื่นๆ ของแบบจำลองเชิงสัมพันธ์ เราไม่จำเป็นต้องใช้แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ก็ได้ แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ยังอาจถูกเรียกว่าเป็น ER model อีกรูปแบบหนึ่ง และในบางครั้งผู้ออกแบบฐานข้อมูลอาจใช้แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ในการออกแบบฐานข้อมูลใน

ระดับแนวคิดเลยก็ได้เนื่องจากมีความคล้ายคลึงกันกับ ER model ทั้งนี้แล้วแต่ความเหมาะสมของระบบและความถนัดของผู้ออกแบบ โดยเราสามารถสร้างรีเลชันและกำหนดเงื่อนไขด้วย SQL ได้เลย ซึ่ง SQL นั้นได้อธิบายไว้อย่างละเอียดในบทที่ 7 ในบทนี้จะอธิบายถึงการเชื่อมโยงและความสัมพันธ์ระหว่าง ER model และแบบจำลองเชิงสัมพันธ์ จึงสรุปได้ว่าเราอาจเลือกวิธีการออกแบบฐานข้อมูลเชิงสัมพันธ์ได้ 2 แบบใหญ่ๆ ดังนี้

การใช้แผนภาพแบบจำลอง 2 ระดับ	**การใช้แผนภาพแบบจำลองข้อมูล 1 ระดับ**
1. เขียน ERD 2. แปลง ERD ให้เป็นโมเดลเชิงสัมพันธ์ด้วย ● แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ หรือ ● เขียนสคีมาของตารางโดยไม่ใช้แผนภาพ 3. สร้างคำสั่ง SQL จากการออกแบบ	1. เขียน ERD หรือ แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์อย่างใดอย่างหนึ่ง 2. สร้างคำสั่ง SQL จากการออกแบบ

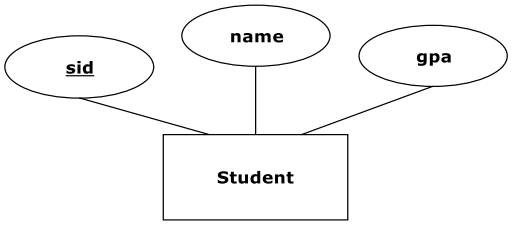
สำหรับหัวข้อต่อไปนี้จะเป็นการแสดงตัวอย่างการสร้างแผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ซึ่งสามารถสร้างจากการวิเคราะห์ความต้องการของผู้ใช้ได้โดยตรง หรือแปลงจาก ER model ได้ดังนี้ สำหรับตัวอย่างการแปลง ER model ให้เป็นแบบจำลองเชิงสัมพันธ์ต่อไปนี้ใช้ตัวอย่างจากเรื่อง ER model ในบทที่ 3

4.5.1 การแปลงเอนทิตีให้เป็นตาราง

เอนทิตี (entity types) ในแบบจำลองเชิงสัมพันธ์แทนได้อย่างตรงไปตรงมาด้วยตารางรีเลชันหรือตารางข้อมูล และแอททริบิวต์ของเอนทิตีคือแอททริบิวต์ของรีเลชัน

ตัวอย่างที่ 4.1 การแปลงเอนทิตีให้เป็นตาราง: มหาวิทยาลัย

ตัวอย่างเอนทิตีในมหาวิทยาลัย ERD ของเอนทิตีนักศึกษา (Student) ประกอบด้วยแอททริบิวต์ รหัสนักศึกษา (sid) ชื่อ-นามสกุล (name) เกรดเฉลี่ย (gpa) ดังนี้



Student	
PK	**sid**
	name gpa

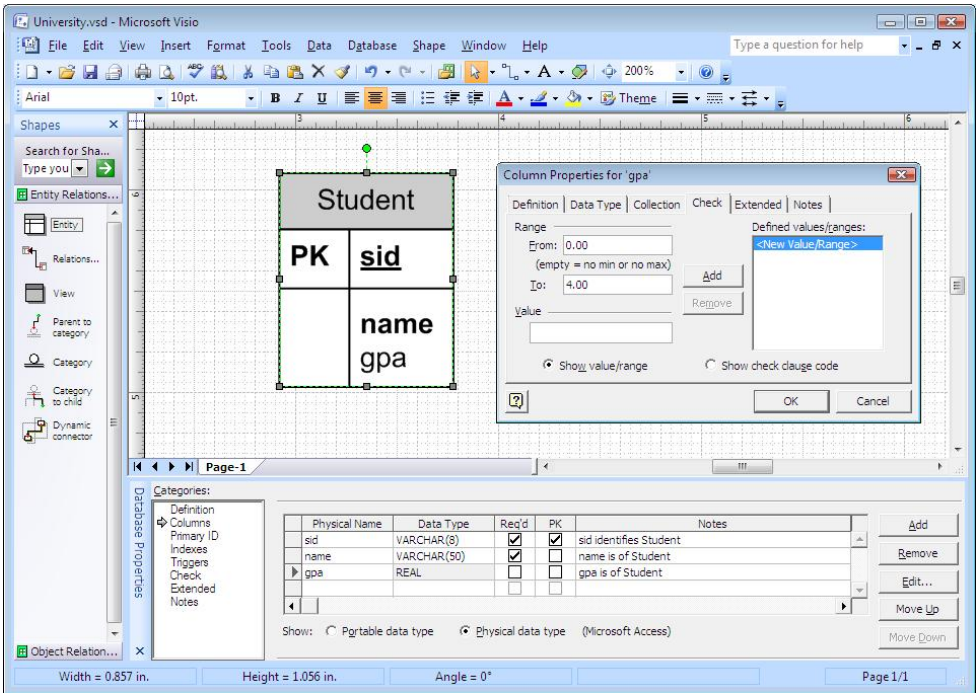
การแปลงจากเอนทิตีสามารถแปลงไปเป็นสคีมาของแบบจำลองเชิงสัมพันธ์ได้อย่างตรงไปตรงมา ทางด้านขวามือเป็นสคีมาของรีเลชัน Student โดยสัญลักษณ์ PK หมายถึงคีย์หลักซึ่งรวมถึงการขีดเส้นใต้แอ

ททริบิวต์ด้วย ตัวหนาหมายถึงแอททริบิวต์นั้นจำเป็นต้องมีข้อมูลและเป็นค่าว่าง (null) ไม่ได้ กรณีตัวอย่างของข้อมูลในรีเลชันสามารถแสดงได้ดังนี้

Student

sid	**name**	**gpa**
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

การสร้างแบบจำลองใดๆ ไม่ว่าจะเป็น ER model หรือแบบจำลองเชิงสัมพันธ์สามารถสร้างได้ด้วย CASE tools ที่ได้กล่าวถึงในบทที่ 1 CASE tool นี้ สามารถใช้เพื่อสร้างแผนภาพแบบจำลองข้อมูลใดๆ กำหนดข้อกำหนดต่างๆ ไว้ในแผนภาพ และสามารถแปลงเป็นคำสั่ง SQL ได้โดยอัตโนมัติ ตัวอย่างต่อไปนี้เป็นการใช้โปรแกรม Microsoft Office Visio ในการวาดแผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ และกำหนดเงื่อนไขบังคับโดเมนให้กับแอททริบิวต์ gpa ให้มีข้อมูลชนิด REAL และมีค่าอยู่ระหว่าง 0.00 ถึง 4.00 เท่านั้น



ภาพที่ 4-3 CASE Tool – Visio ช่วยในการสร้างตารางความสัมพันธ์

การมีภาษาที่มีโครงสร้างสำหรับจัดการระบบฐานข้อมูลเชิงสัมพันธ์นั้นเป็นกฎข้อหนึ่งที่ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ต้องมี เราสามารถใช้ SQL ซึ่งเป็นภาษาที่นิยมในการจัดการระบบจัดการฐานข้อมูลในการสร้างรีเลชันจาก ER model ดังกล่าวข้างต้นได้เช่นเดียวกัน คำสั่ง SQL ที่เกี่ยวข้องในการสร้างรีเลชันและ

กำหนดเงื่อนไขบังคับต่างๆ ยังได้แสดงไว้ในหัวข้อนี้ แต่จะกล่าวถึงเฉพาะจุดสำคัญที่เกี่ยวข้องกับการแปลงการออกแบบฐานข้อมูลระดับแนวคิดเป็นระดับตรรกะเท่านั้น สำหรับรายละเอียดคำสั่ง SQL อธิบายไว้ในบทที่ 7

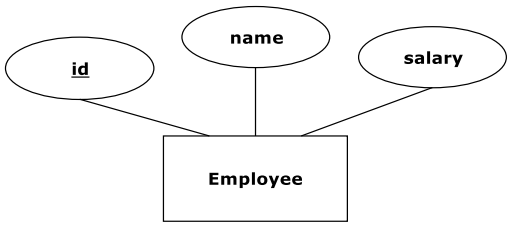
เอนทิตีนักศึกษาสามารถสร้างเป็นรีเลชันด้วยคำสั่ง SQL ดังนี้

```
CREATE TABLE student( sid     VARCHAR(8) NOT NULL,
                      name    VARCHAR(50) NOT NULL,
                      gpa     REAL,
                      PRIMARY KEY (sid))
```

ในที่นี้ student เป็นชื่อรีเลชัน sid, name และ gpa เป็นชื่อแอททริบิวต์ VARCHAR(8), VARCHAR(50) และ REAL เป็นการกำหนดเงื่อนไขบังคับโดเมนของแอททริบิวต์ให้มีชนิดเป็นตัวอักษรขนาดความยาวไม่เกิน 8, 50 และข้อมูลชนิดจำนวนจริงตามลำดับ PRIMARY KEY(sid) เป็นการกำหนดเงื่อนไขบังคับคีย์ ให้แอททริบิวต์ sid เป็นคีย์หลัก สำหรับคำสั่ง NOT NULL เป็นการระบุว่าแอททริบิวต์นั้นๆ ไม่สามารถเป็นค่าว่างได้

ตัวอย่างที่ 4.2 การแปลงเอนทิตีให้เป็นตาราง: บริษัท

ตัวอย่างการแปลงจาก ERD เป็นแบบจำลองเชิงสัมพันธ์เพิ่มเติม เอนทิตีในบริษัททั่วไป ERD ของเอนทิตีพนักงาน (Employee) ประกอบด้วยแอททริบิวต์ รหัสประจำตัวประชาชน (id) ชื่อ-นามสกุล (name) เงินเดือน (salary) ดังนี้



Employee	
PK	id
	name salary

แบบจำลองเชิงสัมพันธ์สามารถแสดงได้ดังแผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ดังรูปด้านขวา และกำหนดเป็นคำสั่ง SQL ได้ดังนี้

```
CREATE TABLE employee( id      VARCHAR(11) NOT NULL,
                       name    VARCHAR(50) NOT NULL,
                       salary  CURRENCY,
                       PRIMARY KEY (id))
```

4.5.2 การแปลงความสัมพันธ์ให้เป็นตาราง

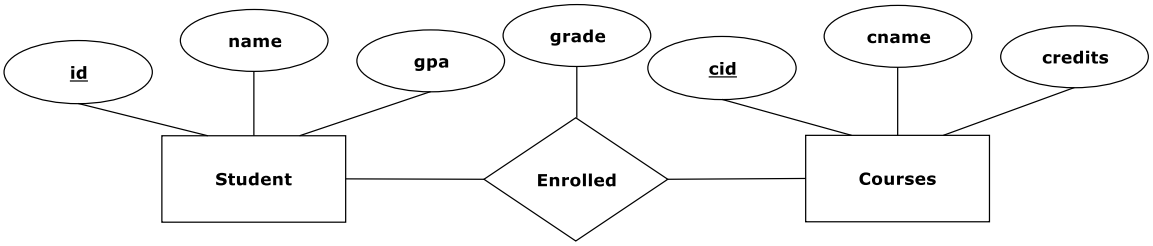
4.5.2.1 ความสัมพันธ์ที่ไม่มีเงื่อนไข

ความสัมพันธ์ที่ไม่มีเงื่อนไขนั้นโดยทั่วไปคือความสัมพันธ์ที่มีคาร์ดินัลลิตีแบบ many-to-many และยังไม่ได้กำหนดเงื่อนไขการมีส่วนร่วม อย่างไรก็ตาม เนื่องจากความสัมพันธ์เป็นความเกี่ยวข้องของตาราง ดังนั้นความสัมพันธ์ที่ไม่มีเงื่อนไขนี้รวมถึงความสัมพันธ์ที่อาจมีเงื่อนไขบังคับภายนอกได้

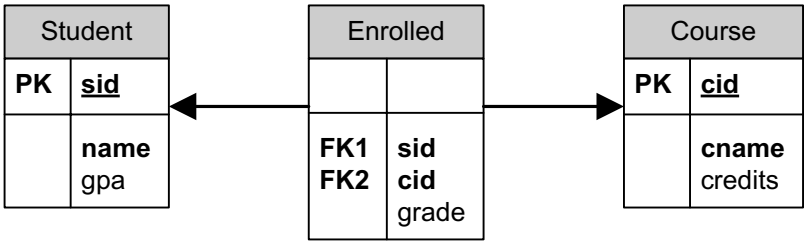
ตัวอย่างที่ 4.3 การแปลงความสัมพันธ์ที่ไม่เงื่อนไขเป็นตารางความสัมพันธ์: มหาวิทยาลัย

ตัวอย่างความสัมพันธ์ในมหาวิทยาลัย ความสัมพันธ์ของนักศึกษา (Student) **ลงทะเบียนเรียน (Enrolled)** รายวิชา (Course) ซึ่งความสัมพันธ์แต่ละความสัมพันธ์ในที่นี้นักศึกษาคนหนึ่งลงทะเบียนรายวิชาหนึ่ง ดังนี้

ER model



Relational database model



SQL

```
CREATE TABLE enrolled ( sid     VARCHAR(8)  NOT NULL,
                        cid     VARCHAR(6) NOT NULL,
                        grade   REAL,
                        FOREIGN KEY (sid) REFERENCES student,
                        FOREIGN KEY (cid) REFERENCES course (cid) )
```

ความสัมพันธ์แบบไม่มีเงื่อนไขนั้นในแบบจำลองเชิงสัมพันธ์แทนด้วยรีเลชันเช่นเดียวกันกับเอนทิตีและแอททริบิวต์ประจำความสัมพันธ์นั้นๆ แทนด้วยแอททริบิวต์ในแบบจำลองเชิงสัมพันธ์เช่นกัน นอกจากนี้ในแบบจำลองเชิงสัมพันธ์ต้องมีการระบุแอททริบิวต์ที่เป็นคีย์หลักจากเอนทิตีที่มีส่วนในความสัมพันธ์นั้นให้เป็นสมาชิกในรีเลชันด้วย ดังนั้นการเพิ่มหรือแก้ไขข้อมูลใดๆ ในรีเลชันของความสัมพันธ์ที่อ้างอิงจากค่าของตารางหรือรีเลชันอื่น ค่านั้นๆ ต้องปรากฏอยู่ในรีเลชันที่อ้างถึงด้วย นำมาซึ่งหลักสำคัญอีกอย่างหนึ่งของการแทนแบบจำลองเชิงสัมพันธ์สำหรับความสัมพันธ์คือการกำหนดเงื่อนไขบังคับภายนอก ซึ่งในแบบจำลองฐานข้อมูลเชิงสัมพันธ์นี้ใช้สัญลักษณ์ FK โดย FK1 เป็นการกำหนดให้ sid ในรีเลชัน Enrolled อ้างอิงถึง sid

ในรีเลชัน Student และ cid ในรีเลชัน Enrolled อ้างอิงถึง cid ของรีเลชัน Course หัวลูกศรของความสัมพันธ์ระหว่างรีเลชันชี้ไปยังตารางที่เป็นเจ้าของคีย์หลัก

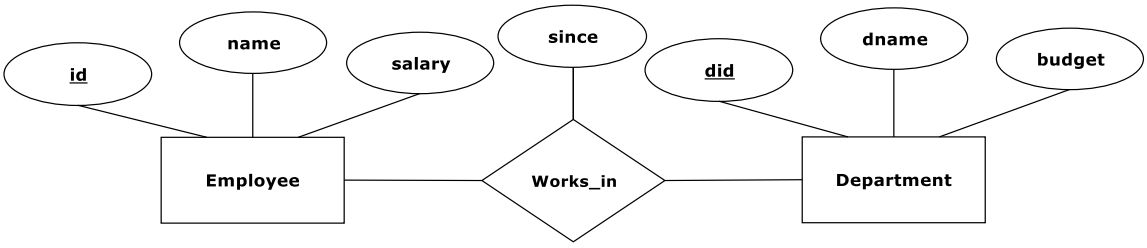
การสร้างรีเลชันจากความสัมพันธ์ Enrolled สอดคล้องกับคำสั่ง SQL ที่ได้แสดงไว้ โดยเป็นการสร้างรีเลชันที่ประกอบไปด้วยแอททริบิวต์ sid, cid และ grade แทนรหัสนักศึกษา รหัสรายวิชาที่นักศึกษาลงทะเบียนเรียน และเกรดที่ได้จากรายวิชานั้นตามลำดับ โดเมนของข้อมูลแอททริบิวต์ในรีเลชันที่เกิดจากความสัมพันธ์นี้ต้องสอดคล้องกับโดเมนของแอททริบิวต์ในเอนทิตีที่อ้างถึง

FOREIGN KEY (sid) REFERENCES student เป็นคำสั่งที่ใช้ในการกำหนดเงื่อนไขบังคับภายนอก หมายความว่าข้อมูลที่จะเพิ่มหรือแก้ไขในแอททริบิวต์ sid ต้องมีปรากฏอยู่ในแอททริบิวต์ชื่อเดียวกันนี้ในรีเลชัน Student สำหรับ FOREIGN KEY (cid) REFERENCES course (cid) มีข้อแตกต่างจากคำสั่งกำหนดเงื่อนไขบังคับแรกตรงที่มีการกำหนดชื่อของแอททริบิวต์ที่อ้างถึงด้วยในกรณีที่แอททริบิวต์ที่อ้างถึงอาจเป็นคนละชื่อกันกับแอททริบิวต์ในรีเลชันที่สร้าง เช่น ในกรณีที่เราอาจใช้ชื่อแอททริบิวต์ course_id แทนรหัสวิชาที่นักศึกษาลงทะเบียน (ไม่ใช่ cid) รหัสวิชานี้อ้างอิงถึงรหัสวิชา cid ในรีเลชัน Course เราสามารถเขียนคำสั่ง SQL ได้เป็น FOREGN KEY (course_id) REFERENCES course (cid) ได้

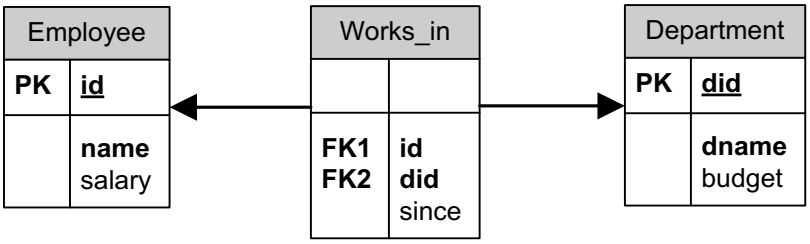
ตัวอย่างที่ 4.4 การแปลงความสัมพันธ์ที่ไม่เงื่อนไขเป็นตารางความสัมพันธ์: บริษัท

ตัวอย่างความสัมพันธ์ในบริษัททั่วไป ความสัมพันธ์ของพนักงาน (Employee) **ทำงานใน (Works_in)** แผนก (Department) ซึ่งความสัมพันธ์แต่ละความสัมพันธ์ในที่นี้คือพนักงานคนหนึ่งทำงานให้แผนกหนึ่ง ดังนี้

ER model



Relational Database Model



SQL

```
CREATE TABLE works_in (id      VARCHAR(11)  NOT NULL,
                       did     INTEGER NOT NULL,
                       since   DATE,
                       FOREIGN KEY (id) REFERENCES employee,
                       FOREIGN KEY (did) REFERENCES department)
```

สำหรับความสัมพันธ์แบบไตรภาคีสามารถสร้างได้เช่นเดียวกับความสัมพันธ์แบบทวิภาคี เพียงเพิ่มการอ้างอิงไปยังอีกเอนทิตีหนึ่งเท่านั้นเอง

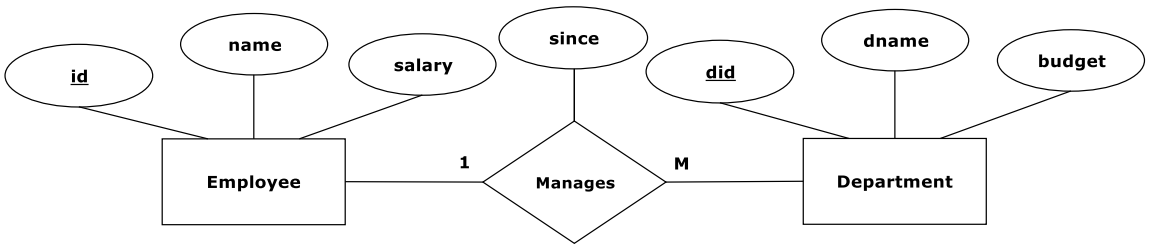
4.5.2.2 ความสัมพันธ์ที่มีเงื่อนไขบังคับคีย์ หรือ มีคาร์ดินัลลิตีแบบ one-to-many

เงื่อนไขบังคับคีย์ในความสัมพันธ์หมายถึงการที่คีย์หลักของเอนทิตีหนึ่งจะมีค่าอยู่ในความสัมพันธ์ได้เพียงครั้งเดียว แสดงดังตัวอย่างได้ดังนี้

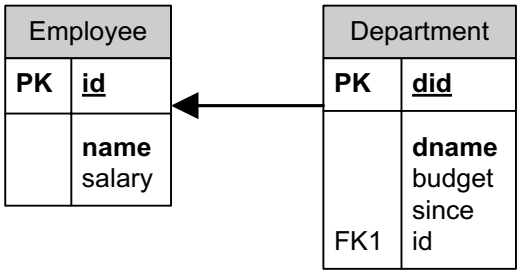
ตัวอย่างที่ 4.5 การแปลงความสัมพันธ์ที่มีเงื่อนไขบังคับคีย์เป็นตารางความสัมพันธ์

แผนกหนึ่งแผนกจะมีผู้จัดการได้เพียงคนเดียว รหัสแผนก 1 แผนกจะปรากฏอยู่ในความสัมพันธ์การจัดการได้ 1 ครั้งเท่านั้น ในกรณีที่พนักงาน 1 คนสามารถเป็นผู้จัดการได้หลายแผนกเราจะพบว่าเงื่อนไขบังคับคีย์ใช้สำหรับความสัมพันธ์ในรูปแบบ one-to-many นั่นเอง แบบจำลองเชิงสัมพันธ์จาก ER model ของการจัดการแผนกสามารถแสดงได้ดังต่อไปนี้

ER model



Relational Database Model



SQL

```
CREATE TABLE department (did    INTEGER NOT NULL,
                         dname  VARCHAR(50),
                         budget CURRENCY,
                         since  DATE,
                         id     VARCHAR(11),
                         PRIMARY KEY (did),
                         FOREIGN KEY (id) REFERENCES employee (id) )
```

ความสัมพันธ์แบบ one-to-many สามารถแทนได้ด้วยรีเลชันเช่นเดียวกับความสัมพันธ์แบบไม่มีเงื่อนไข แต่เราสามารถลดรีเลชันได้อีก โดยไม่แทนความสัมพันธ์ด้วยรีเลชัน ด้วยการยุบความสัมพันธ์นั้นแล้วนำแอททริบิวต์ของความสัมพันธ์ไปรวมเข้ากับเอนทิตีในฝั่ง *N* พิจารณาจากข้อกำหนดของความสัมพันธ์จัดการ (Manages) แผนกทุกแผนกมีผู้จัดการได้ 1 คนอยู่แล้ว ดังนั้นเราสามารถเพิ่มแอททริบิวต์ผู้จัดการในรีเลชันแผนกได้เลย ในแผนภาพคือ id ซึ่งอ้างถึงรหัสพนักงานที่จะเป็นผู้จัดการในแผนกนั้น และนำแอททริ

บิวต์ since มารวมในรีเลชันแผนกเช่นเดียวกัน การรวมรีเลชันเช่นนี้จะลดความยุ่งยากและทำให้การใช้งานรวดเร็วขึ้นในการแสดงข้อมูลการจัดการของแผนก (ในบางกรณีทำให้ประหยัดเวลาประมวลผลเป็นอย่างมาก) โดยเราไม่จำเป็นต้องเชื่อมรีเลชันถึง 3 รีเลชันเข้าด้วยกัน อย่างไรก็ตามการลดรีเลชันสำหรับความสัมพันธ์แบบ one-to-many อาจมีผลเสีย กล่าวคือหากข้อกำหนดทางธุรกิจไม่ได้ระบุว่าแผนกทุกแผนกต้องมีผู้จัดการ ข้อมูลผู้จัดการในตารางแผนกจะว่างอยู่ (ซึ่งต้องแทนด้วย NULL) หากเป็นข้อมูลอื่นที่มีจำนวนมากกว่านี้ การยุบรวมรีเลชันจะทำให้เกิดค่าว่างมากมายและเสียเนื้อที่จัดเก็บข้อมูล

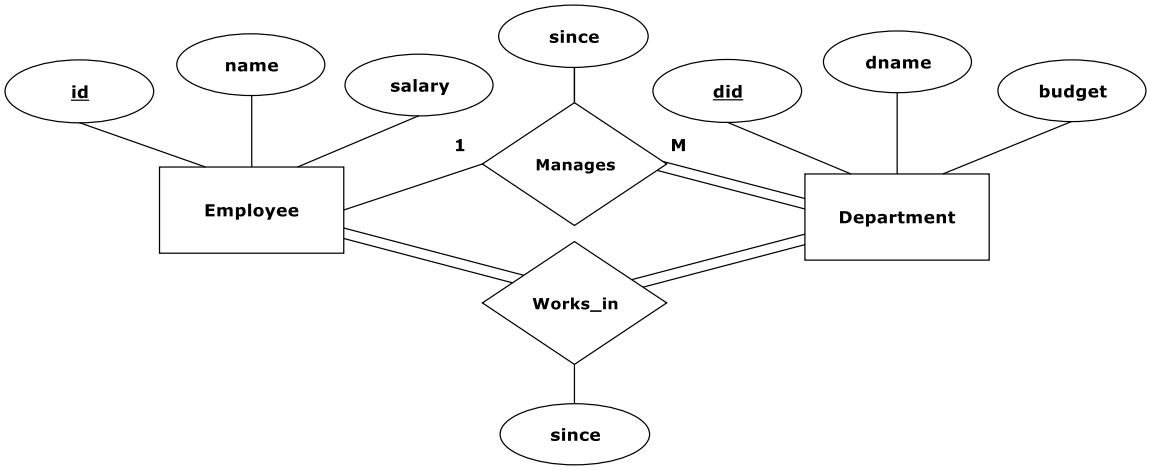
4.5.2.3 ความสัมพันธ์ที่มีเงื่อนไขบังคับการมีส่วนร่วม

การกำหนดการมีส่วนร่วมแบบทั้งหมด (total participation) สามารถกระทำได้โดยการบังคับให้แอททริบิวต์ที่มีส่วนร่วมในความสัมพันธ์ไม่สามารถเป็นค่าว่างได้

ตัวอย่างที่ 4.6 การแปลงความสัมพันธ์ที่มีเงื่อนไขบังคับการมีส่วนร่วมเป็นตารางความสัมพันธ์

ความสัมพันธ์จัดการ (Manages) ที่ระบุเงื่อนไขเพิ่มเติมว่านอกจากแต่ละแผนกจะมีผู้จัดการได้เพียง 1 คนแล้ว ทุกแผนกจะต้องมีผู้จัดการ ดังนั้นแต่ละทูเพิลของแผนก จะต้องระบุรหัสผู้จัดการด้วย

ER model



Relational database model

Employee	
PK	id
	name salary

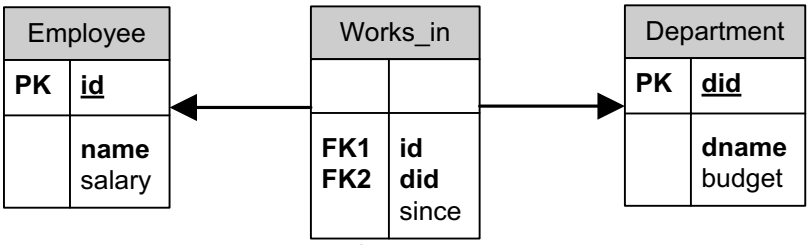
Department	
PK	did
	dname budget since
FK1	id

SQL

```
CREATE TABLE department (did    INTEGER NOT NULL,
                         dname  VARCHAR(50),
                         budget CURRENCY,
                         since  DATE,
                         id     VARCHAR(11) NOT NULL,
                         PRIMARY KEY (did),
                         FOREIGN KEY (id) REFERENCES employee (id) )
```

แผนกทุกแผนกจำเป็นต้องมีผู้จัดการ สามารถรองรับได้ด้วยเงื่อนไขบังคับการมีส่วนร่วมทั้งหมด การบังคับให้ทุกๆ ทูเพิลของแผนกมีรหัสพนักงานที่เป็นผู้จัดการกำหนดได้โดยการใช้ตัวหน้าในแบบจำลองฐานข้อมูลเชิงสัมพันธ์และโดยการใช้คำสั่ง NOT NULL สำหรับแอททริบิวต์ id ในตาราง department ด้วย SQL ดังที่แสดง

สำหรับความสัมพันธ์ที่แทนด้วยรีเลชันโดยไม่ได้ทำการยุบรีเลชัน (ที่กล่าวถึงในหัวข้อ 4.5.1) หรือรีเลชันที่สร้างจากความสัมพันธ์ที่ไม่มีเงื่อนไข (many-to-many) นั้นไม่สามารถที่จะกำหนดเงื่อนไขบังคับได้จากคำสั่งพื้นฐานในการสร้างรีเลชันด้วย SQL เราต้องใช้ SQL ชนิดที่มีความซับซ้อนกว่านี้ในรูปแบบที่เรียกว่า assertion ซึ่งจะได้กล่าวถึงในบทที่ 7 ซึ่งรูปต่อไปนี้แสดงถึงความสัมพันธ์ที่ต้องอาศัย assertion ดังกล่าว



เราสามารถระบุได้ว่าทุกๆ ทูเพิลที่เกิดขึ้นในความสัมพันธ์พนักงานทำงานในแผนก (Works_in) จำเป็นต้องอ้างถึงรีเลชันพนักงาน และแผนก ตลอดจนมีค่าว่างไม่ได้ แต่ไม่สามารถบังคับได้ว่ารหัสพนักงานทุกคนต้องปรากฏในความสัมพันธ์ Works_in ด้วยในกรณีที่มีการกำหนดว่าพนักงานทุกคนต้องทำงานให้แผนกใดแผนกหนึ่ง

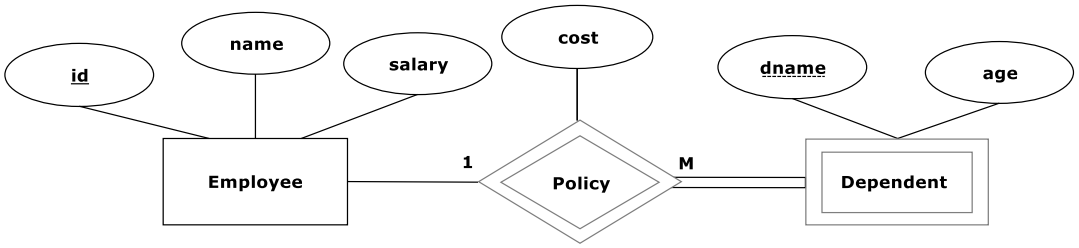
4.5.3 การแปลงเอนทิตีอ่อนแอ

เอนทิตีอ่อนแอนั้นจะอยู่ในรูปแบบของความสัมพันธ์แบบ one-to-many และการมีส่วนร่วมทั้งหมดตลอด ดังนั้นจึงมีการสร้างรีเลชันเช่นเดียวกับในตัวอย่างหัวข้อ 4.5.2.3 แต่กำหนดการจัดการเพิ่มเติมกรณีที่ข้อมูลในตารางหลักที่ถูกอ้างอิงถูกลบทิ้งได้

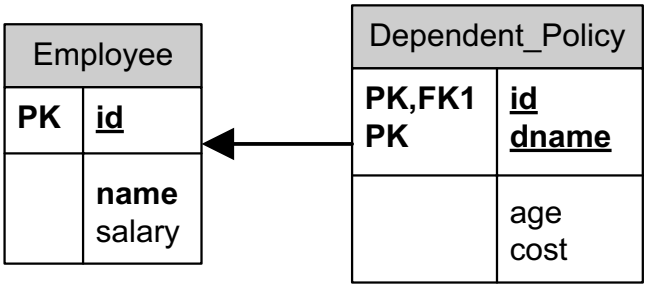
ตัวอย่างที่ 4.7 การแปลงเอนทิตีอ่อนแอเป็นตารางความสัมพันธ์

พนักงานซื้อกรมธรรม์สุขภาพให้กับบุตร/ธิดา บุตร/ธิดาแต่ละคนต้องมีรหัสพนักงานกำกับไว้ว่าเป็นผู้ปกครองเสมอ ดังนั้นเราสามารถยุบความสัมพันธ์ Policy เข้ากับเอนทิตี Dependent ได้ รวมถึงแอททริบิวต์ cost ที่นำมาบรรจุไว้ในเอนทิตี Dependent มีลักษณะเดียวกันกับการกำหนดเงื่อนไขบังคับการมีส่วนร่วมทั้งหมด แต่ทูเพิลในเอนทิตีอ่อนแอจะไม่จำเป็นต้องถูกเก็บไว้หากข้อมูลที่อ้างถึงในเอนทิตีหลักถูกลบทิ้ง เช่น เมื่อพนักงานคนใดลาออก ข้อมูลของบุตร/ธิดาที่อยู่ในเอนทิตีอ่อนแอจะถูกลบทิ้งด้วย ซึ่งแทนด้วยแบบจำลองเชิงสัมพันธ์ และ SQL ดังนี้

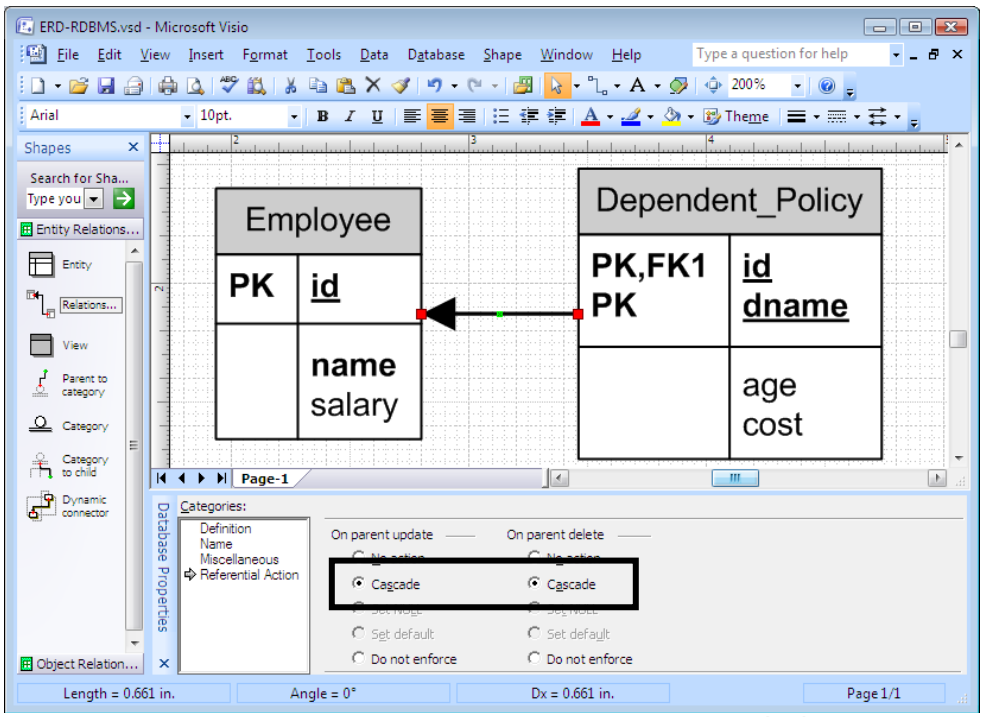
ER model



Relational database model



เอนทิตีอ่อนแอ คือเอนทิตีต้องอาศัยคีย์หลักจากตารางอ้างอิงรวมกับแอททริบิวต์ใดแอททริบิวต์หนึ่งในเอนทิตีอ่อนแอเพื่อกำหนดเป็นคีย์หลัก ในที่นี้ Dependent_Policy เป็นรีเลชันที่เกิดจากเอนทิตีอ่อนแออ้างอิงรหัสพนักงาน (id) ยังตาราง Employee รวมกับแอททริบิวต์ dname เราจึงกำหนดให้ทั้ง id และ dname เป็น PK สำหรับ id เป็น FK อ้างอิงจากตาราง Employee สำหรับการกำหนดให้ข้อมูลในตาราง Dependent_Policy สามารถลบทิ้งหรือปรับปรุงตามข้อมูลในตารางหลักเรากำหนดได้ด้วย CASE tool ดังรูปต่อไปนี้ ในที่นี้ระบุว่าหากมีการปรับปรุงหรือลบข้อมูลในตารางหลัก Employee (id) ข้อมูลในเอนทิตีอ่อนแอ Dependent_Policy ซึ่งได้แก่รหัสพนักงานต้องถูกปรับปรุงตามหรือลบทิ้งทั้งทูเพิล โดยใช้การวิธี Cascade ดังภาพต่อไปนี้



ภาพที่ 4-4 CASE Tool – Visio ช่วยในการกำหนดการจัดการกับการดำเนินการที่มีเงื่อนไขบังคับคีย์ภายนอก

การสร้างคอลัมน์ด้วย SQL มีลักษณะเหมือนการสร้างรีเลชันทั่วไป เนื่องจากตารางนี้มีคีย์หลักแบบคีย์ประกอบซึ่งประกอบด้วย id และ dname จึงใช้คำสั่ง PRIMARY KEY (id, dname) สำหรับการกำหนดให้ระเบียนที่มีค่าขึ้นอยู่กับข้อมูลในตารางหลัก เราใช้คำสั่ง ON DELETE CASCADE สำหรับกำหนดให้ข้อมูลลบทิ้งตามตารางหลัก

4.5.4 การแปลงลำดับชั้นของคลาส

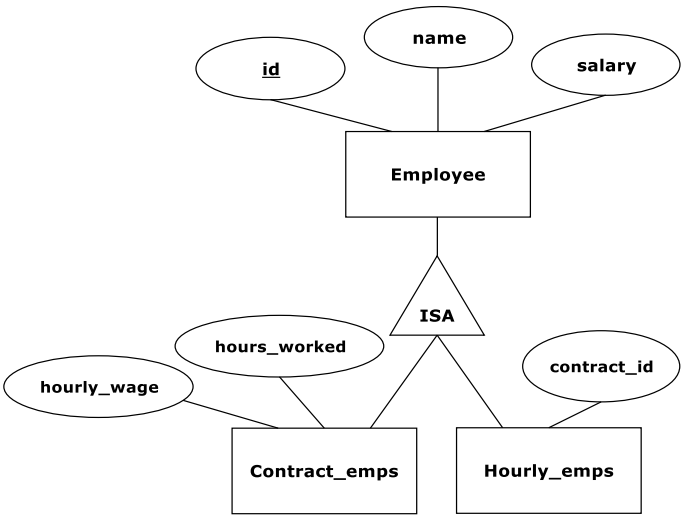
การแปลงลำดับชั้นของคลาสให้อยู่ในรูปแบบของแบบจำลองเชิงสัมพันธ์โดยทั่วไปทำได้ 2 วิธีคือการสร้างรีเลชันของคลาสหลัก จากนั้นทำการสร้างรีเลชันของคลาสที่สืบทอดและอ้างอิงไปยังรีเลชันของคลาสหลัก รีเลชันที่สร้างจากคลาสสืบทอดสามารถบรรจุแอททริบิวต์เฉพาะของคลาสตนเองเพิ่มได้นอกจากแอททริบิวต์ที่คงอยู่ในคลาสหลักแล้ว อีกวิธีหนึ่งคือการสร้างรีเลชันที่เกิดจากคลาสที่สืบทอดทั้งหมดโดยไม่ต้องสร้างคลาสหลัก แอททริบิวต์ที่มีอยู่ในคลาสหลักก็จะมีบรรจุอยู่ในทั้งรีเลชันของคลาสย่อยทั้งหมด

ตัวอย่างที่ 4.8 การแปลงลำดับชั้นของคลาสเป็นตารางความสัมพันธ์

พิจารณาลำดับชั้นของคลาสพนักงานแบบเซ็นสัญญาทำงานและพนักงานรายชั่วโมงจากตัวอย่างในบทที่ 3 หัวข้อ 3.4.4 สามารถแปลงเป็นรีเลชันได้ดังนี้

แบบที่ 1 คือการสร้างรีเลชันจากคลาสหลักและคลาสที่สืบทอดทั้งหมด คลาสหลักบรรจุแอททริบิวต์ที่คลาสย่อยมีเช่นเดียวกัน ทำให้ข้อมูลที่จัดเก็บไม่ซ้ำซ้อน นอกจากนี้แอททริบิวต์พิเศษของคลาสย่อยแต่ละคลาสที่ไม่ปรากฏในอีกคลาสหนึ่งก็สามารถแยกเก็บไว้คนละรีเลชันได้ โดยทั่วไปการสร้างรีเลชันในลักษณะนี้สามารถกระทำได้และความเหมาะสม แต่มีความซับซ้อนในการจัดการระดับหนึ่งเนื่องจากต้องมีการรวมตารางหลักและตารางจากคลาสสืบทอดเพื่อให้ได้ข้อมูลของพนักงานคนใดๆ ให้ครบถ้วน

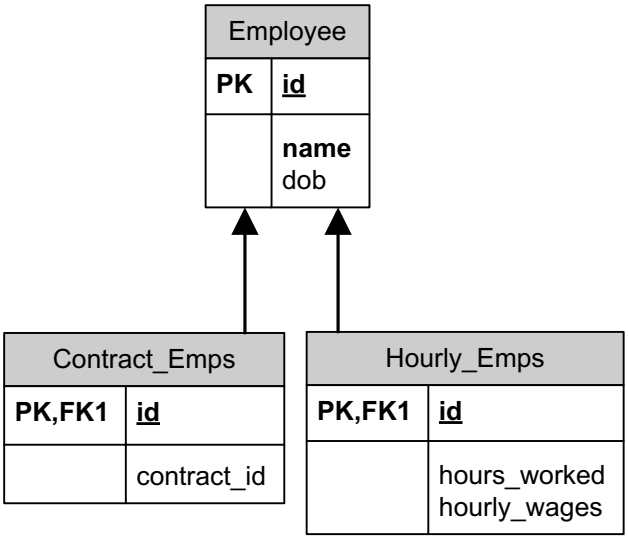
ER model



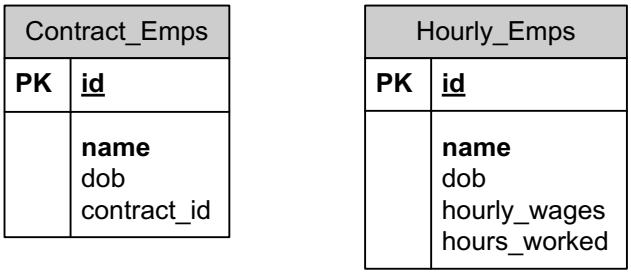
Relational database model

แบบจำลองเชิงสัมพันธ์ของลำดับชั้นของคลาสสามารถสร้างได้ 2 รูปแบบดังนี้

แบบที่ 1



แบบที่ 2



ในขณะที่แบบที่ 2 เป็นการจัดเก็บแอททริบิวต์ที่เหมือนกันในทั้ง 2 ตาราง ได้แก่มีทั้งรหัสพนักงาน (id) ชื่อ (name) และวันเดือนปีเกิด (dob) และเพิ่มแอททริบิวต์ประจำสำหรับรีเลชันที่แทนพนักงานแต่ละ

ประเภท การประยุกต์ในลักษณะนี้มีความสะดวกมากกว่า แต่อาจไม่รองรับการทำงานบางประเภทเช่นในกรณีที่พนักงาน 1 คนเป็นพนักงานที่ทำงานทั้งแบบเซ็นสัญญาและแบบรายชั่วโมง การจัดเก็บข้อมูลจะมีความซ้ำซ้อน หรือในกรณีที่พนักงานไม่ได้เป็นประเภทใดประเภทหนึ่งเลยก็จะไม่สามารถจัดเก็บข้อมูลได้ การสร้างรีเลชันด้วย SQL ใช้คำสั่งเดียวกันกับการสร้างรีเลชัน และการใช้เงื่อนไขบังคับอื่นๆ เพียงแต่ต้องสร้างรีเลชันให้สอดคล้องกับคลาสเท่านั้น

4.6 พจนานุกรมข้อมูล (Data Dictionary)

แบบจำลองเชิงสัมพันธ์ที่ได้สร้างขึ้นถูกกำกับด้วยเอนทิตีและแอททริบิวต์ต่างๆ ซึ่งเรากำหนดคำสั้นๆ ให้กับองค์ประกอบเหล่านั้น เรามีความจำเป็นต้องบันทึกและอธิบายว่าองค์ประกอบนั้น หมายถึงอะไร พจนานุกรมข้อมูล (Data Dictionary) เป็นเอกสารสำคัญของ DBMS ซึ่งอธิบายรายละเอียดของฐานข้อมูลที่ DBMS บรรจุอยู่ เป็นเอกสารเก็บคำอธิบายรายละเอียดต่างๆ ของฐานข้อมูลในลักษณะของเมทาดาตา (metadata) ประกอบด้วยสคีมาของฐานข้อมูลระดับแนวคิด ระดับภายนอก และระดับภายใน รายละเอียดเกี่ยวกับความมั่นคงปลอดภัยต่างๆ ข้อกำหนดต่างๆ เงื่อนไขบังคับบูรณภาพต่างๆ ดังนั้นพจนานุกรมต้องถูกสร้างขึ้นตั้งแต่การออกแบบ ERD แล้ว พจนานุกรมข้อมูลเปรียบได้กับพจนานุกรมศัพท์ทั่วไปที่อธิบายความหมายของคำ แต่ในกรณีนี้อธิบายความหมายและรายละเอียดของข้อมูล

ข้อมูลพื้นฐานที่ควรบันทึกไว้ในพจนานุกรมข้อมูลได้แก่ รายละเอียดของรีเลชัน/ตารางความสัมพันธ์ ซึ่งประกอบด้วย ชื่อรีเลชัน ชื่อแอททริบิวต์ โดเมน เงื่อนไขบังคับคีย์หลัก เงื่อนไขบังคับบูรณภาพอื่นๆ และคำอธิบายอื่นๆ ในบริบทการใช้งาน นอกจากนี้วัตถุอื่นๆ ที่สร้างขึ้นทั้งหมดในฐานข้อมูลก็ควรบนทึกไว้ในพจนานุกรมด้วย เช่น วิว และ index เป็นต้น ตารางต่อไปนี้เป็นตัวอย่างของพจนานุกรมของฐานข้อมูลมหาวิทยาลัย ซึ่งพจนานุกรมมีหลายรูปแบบ การกำหนดคอลัมน์ข้อมูลที่จะจัดเก็บคำอธิบายของข้อมูลสามารถปรับได้ตามความเหมาะสม

Relation	Attribute	Data Type	Primary Key	Foreign Key	Reference	Description & Remarks
student	-	-	-	-	-	ตารางข้อมูลนักศึกษา
student	sid	CHAR(8)	YES	-		รหัสนักศึกษา ขึ้นต้นด้วย B คือนักศึกษาป.ตรี M คือนักศึกษาป.โท และ D คือนักศึกษา ป.เอก
student	name	VARCHAR(50)	-	-		ชื่อนักศึกษา
student	login	VARCHAR(50)	-	-		ชื่อที่ใช้ในการเข้าใช้ระบบ
student	gpa	NUMBER	-	-		เกรดเฉลี่ยรวมของนักศึกษา
faculty	-	-	-	-	-	ตารางข้อมูลคณาจารย์
faculty	fid	CHAR(7)	-	YES	-	รหัสอาจารย์
faculty	fname	CHAR(30)	YES	-	-	ชื่ออาจารย์
faculty	sal	CHAR(3)	-	-	-	เงินเดือนอาจารย์
…	…	…	…	…	…	…

ภาพที่ 4-5 ตัวอย่างพจนานุกรมข้อมูลของฐานข้อมูลมหาวิทยาลัย

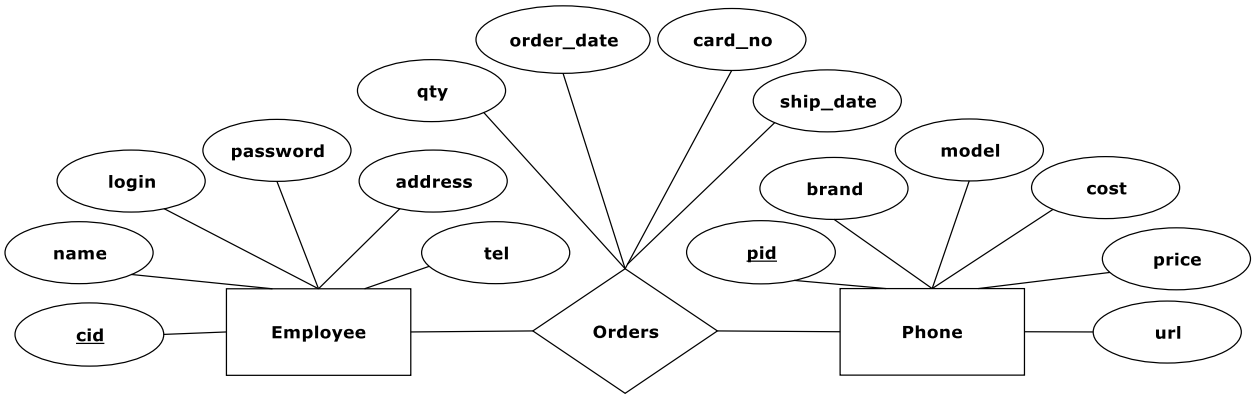
พจนานุกรมสามารถถูกสร้างขึ้นโดยอัตโนมัติด้วย DBMS ซึ่งเรียกว่า Active Data Dictionary และพจนานุกรมที่สร้างขึ้นโดยผู้ออกแบบฐานข้อมูลหรือ DBA เรียกว่า Passive Data Dictionary

4.7 กรณีศึกษา

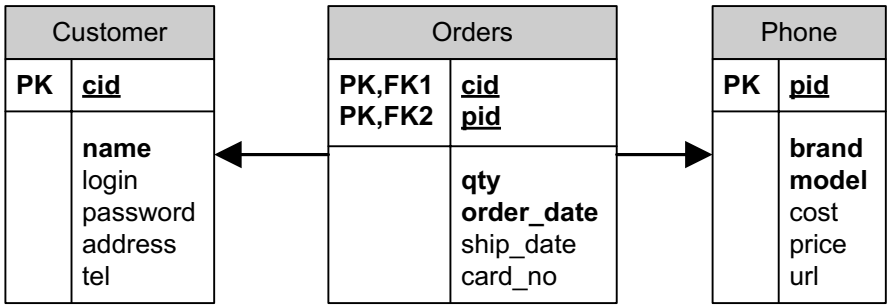
ตัวอย่างกรณีศึกษาในบทที่ 3 สามารถนำมาสร้างเป็นแบบจำลองเชิงสัมพันธ์โดยใช้แผนภาพแบบจำลองฐานข้อมูลเชิงสัมพันธ์ที่สัมพันธ์กับ ER model โดยใช้หลักการแปลงจาก ER model เป็นแบบจำลองเชิงสัมพันธ์ดังได้อธิบายในบทนี้ การสร้างแบบจำลองเชิงสัมพันธ์ที่ซับซ้อนต้องเริ่มจากการค่อยๆ แปลงส่วนใดส่วนหนึ่งของ ER model จากนั้นค่อยๆ ขยายออกไปยังรีเลชันที่เกี่ยวข้องจนเสร็จสมบูรณ์ ซึ่งแสดงผลลัพธ์ของการแปลงได้ดังต่อไปนี้

<u>7-Elephant</u>

ER Model

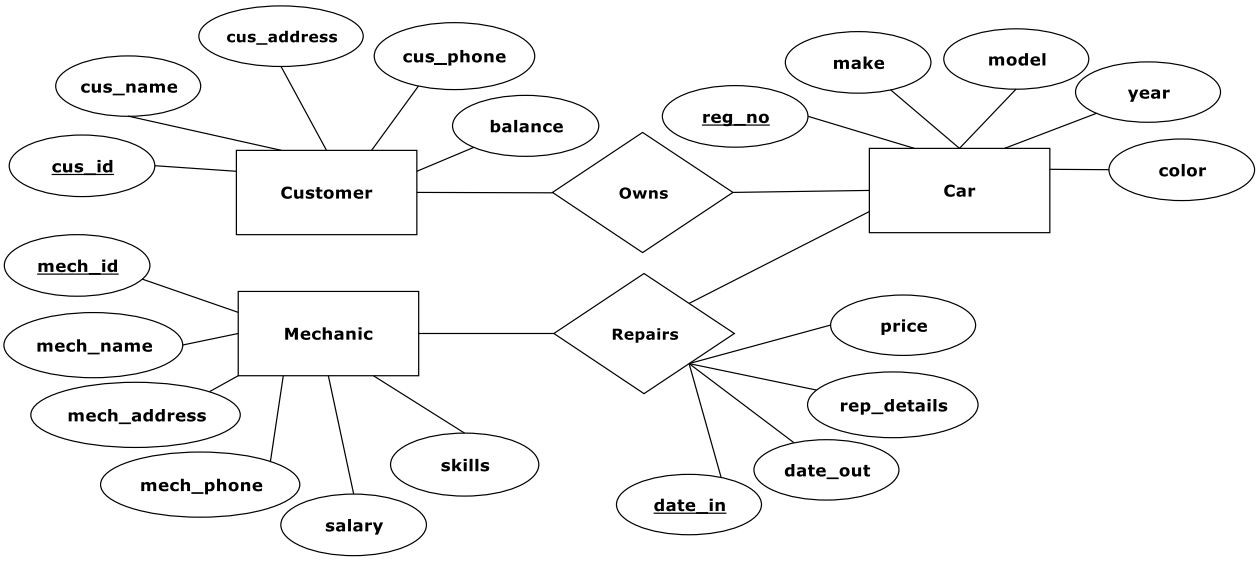


Relational database model

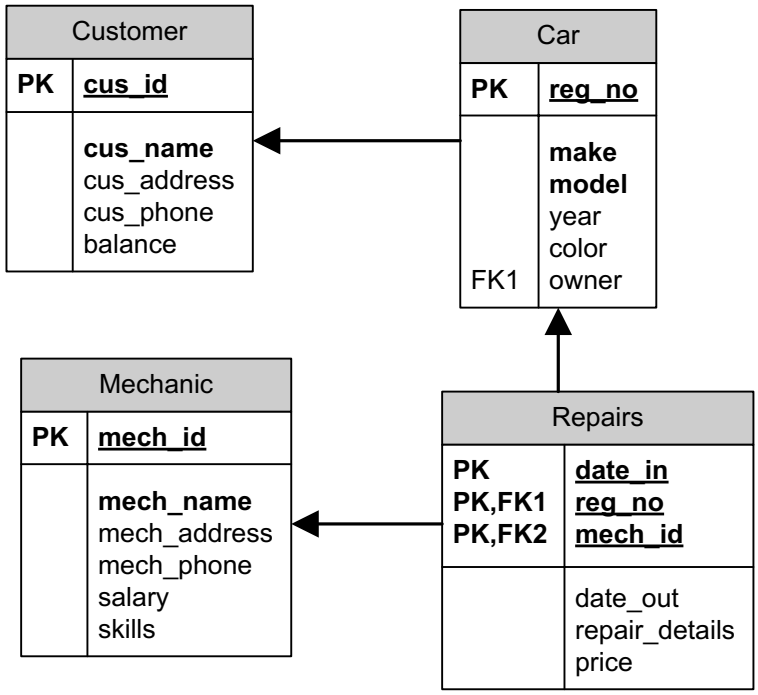


<u>อู่เฉิดฉาย</u>

ER model

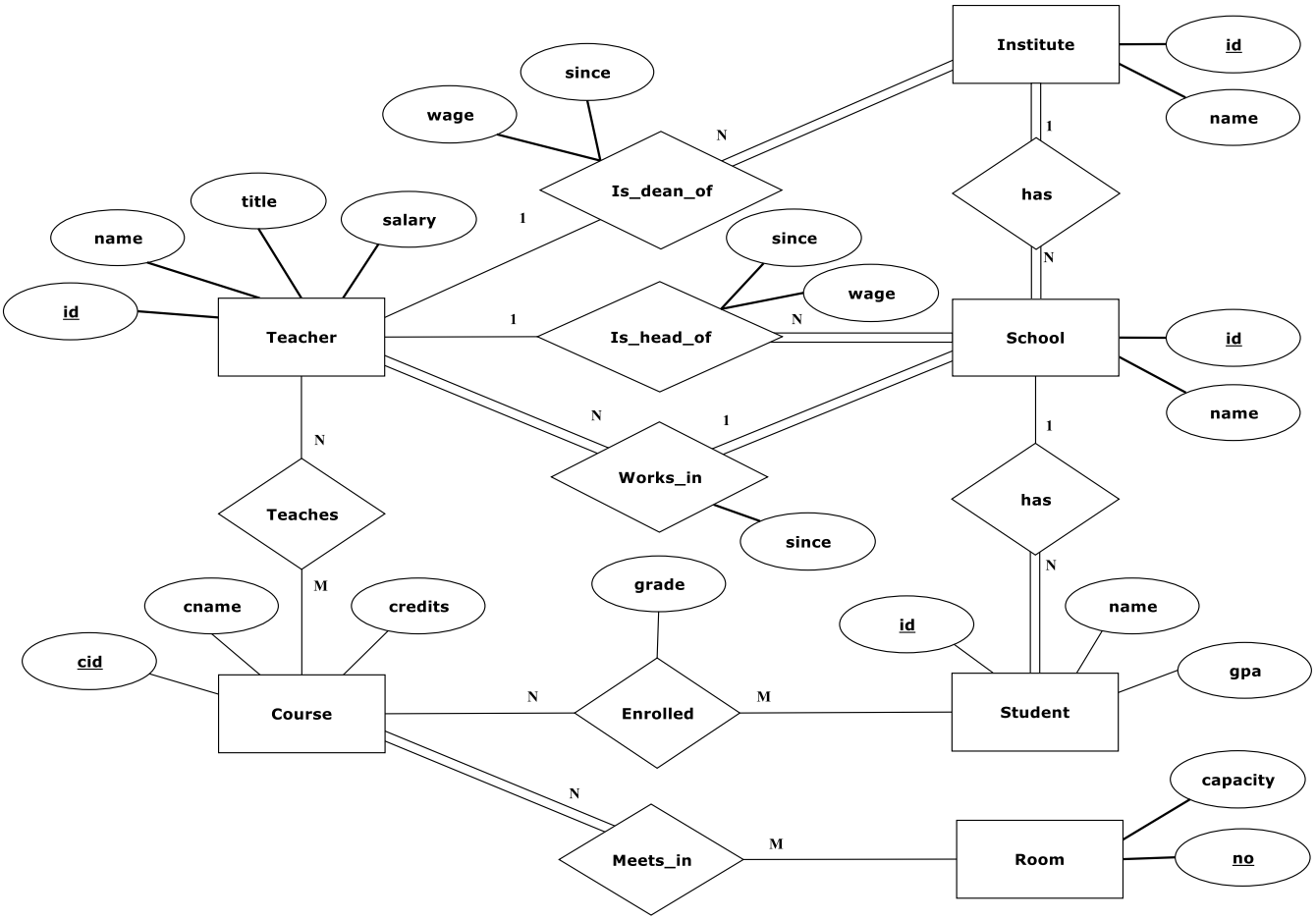


Relational database model

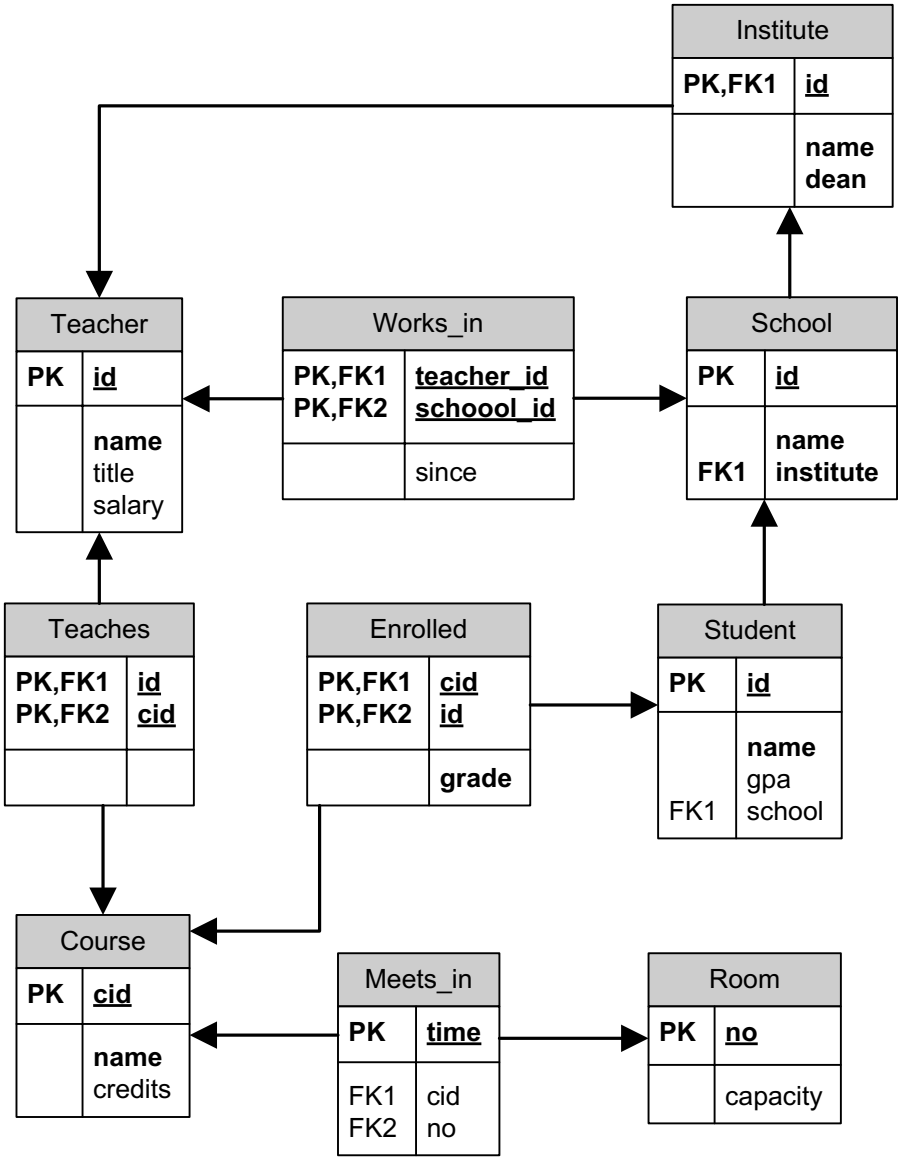


<u>มหาวิทยาลัยเทคโนโลยีสุรบุรุษ</u>

ER model



Relational database model



4.8 บทสรุป

แบบจำลองข้อมูลเชิงสัมพันธ์เป็นแบบจำลองข้อมูลของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ซึ่งเป็นชนิดของระบบจัดการฐานข้อมูลที่ได้รับความนิยมแพร่หลายมากที่สุดในปัจจุบัน ก่อนที่จะมีการใช้แบบจำลองเชิงสัมพันธ์ ได้มีการนำเสนอและประยุกต์ใช้งานแบบจำลองข้อมูลมาแล้วหลายแบบในอดีตเพื่อสร้างฐานข้อมูลที่มีประสิทธิภาพ E.F. Codd เป็นผู้นำเสนอแบบจำลองเชิงสัมพันธ์ซึ่งจะต้องรองรับกฎ 12 ข้อของ Codd แบบจำลองเชิงสัมพันธ์แทนข้อมูลเป็นรีเลชันหรือตารางความสัมพันธ์ซึ่งมีรูปแบบที่เข้าใจง่าย มีลักษณะเช่นเดียวกับข้อมูลที่ใช้กันโดยทั่วไปในรูปของตาราง เราจะต้องทำการกำหนดสคีมาหรือเค้าร่างของตารางความสัมพันธ์เพื่อจัดเก็บข้อมูลหรือกรณีตัวอย่าง

ทูเพิลคือระเบียน แถว หรือเรคอร์ดของข้อมูล ในขณะที่ฟิลด์คือ แอททริบิวต์หรือคอลัมน์ของข้อมูลตาราง แอททริบิวต์แต่ละแอททริบิวต์จะถูกกำหนดเขตหรือโดเมนของค่า ดีกรีของรีเลชันคือจำนวนแอททริบิวต์ และคาร์ดินัลลิตีคือจำนวนแถวของรีเลชัน การอ้างอิงถึงข้อมูลทูเพิลใดๆ จะต้องเข้าถึงได้เฉพาะเจาะจงด้วยคีย์ ซึ่งคีย์นั้นมีหลายประเภท ได้แก่คีย์หลัก คีย์ประกอบ คีย์คู่แข่ง คีย์ภายนอก ฯลฯ แบบจำลองเชิงสัมพันธ์สามารถรองรับเงื่อนไขบังคับบูรณภาพของข้อมูล เพื่อบังคับให้การจัดการข้อมูลในรีเลชันมีความถูกต้องอยู่เสมอได้

เราสามารถนำแบบจำลองข้อมูลระดับแนวคิดมาแทนด้วยแบบจำลองเชิงตรรกะได้โดยนำเอนทิตีของ ERD และความสัมพันธ์ของ ERD มาแทนด้วยรีเลชัน จากนั้นกำหนดแอททริบิวต์ของทั้งเอนทิตีและความสัมพันธ์ให้เป็นแอททริบิวต์สมาชิกในความสัมพันธ์ จากนั้นกำหนดเงื่อนไขบังคับบูรณภาพต่างๆ จาก ERD และข้อกำหนดอื่นๆ ของระบบ เราสามารถกำหนดวิวหรือมุมมองของผู้ใช้ครอบรีเลชันหลักที่บรรจุอยู่ในฐานข้อมูลได้ เพื่อความมั่นคงปลอดภัยในการใช้งานฐานข้อมูล รวมถึงการคงความเป็นอิสระของข้อมูล เมื่อทำการออกแบบฐานข้อมูลระดับตรรกะนี้แล้วเสร็จ เราจะสามารถนำไปประยุกต์ในการพัฒนาฐานข้อมูลเพื่อใช้งานต่อไปในระบบจัดการฐานข้อมูล ซึ่งข้อมูลต่างๆ ในผลลัพธ์ของการออกแบบนี้จะต้องถูกจัดเก็บคำอธิบายโดยละเอียดเพื่อใช้อ้างอิงในโอกาสต่อไปด้วยพจนานุกรมข้อมูล

4.9 แบบฝึกหัดท้ายบท

1. พิจารณา instance ของ Students relations ดังต่อไปนี้ (Ramakrishnan & Gehrke, 2003)

FIELDS (ATTRIBUTES, COLUMNS)

Field names

TUPLES (RECORDS, ROWS)

sid	*name*	*login*	*age*	*gpa*
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53650	Smith	smith@math	19	3.8
53831	Madayan	madayan@music	11	1.8
53832	Guldu	guldu@music	12	2.0

ให้ระบุว่า attribute ใดบ้างที่ไม่เป็น candidate key โดยพิจารณาจาก instance ที่ให้มาเท่านั้น

2. ให้ออกแบบ Relational Database Logical Model ของบริษัท Art Gallery รายขายยาหมอตี๋ และ Mummy Records ในแบบฝึกหัดท้ายบทที่ 3 พร้อมทั้งเขียนพจนานุกรมข้อมูลด้วย

3. ให้ออกแบบ Relational Database Logical Model ของโครงงาน “เช่า-ฉัน-สิ” ในภาคผนวก พร้อมทั้งเขียนพจนานุกรมข้อมูลด้วย

บรรณานุกรมและเอกสารอ้างอิงประจำบท

ชนวัฒน์ ศรีสอ้าน. (2542). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

รวิวรรณ เทนอิสสระ. (2543). **ฐานข้อมูลและการออกแบบ.** กรุงเทพฯ : ซีเอ็ดยูเคชั่น.

วราภรณ์ โกวิทวรางกูร. (2543). **ระบบฐานข้อมูลและการออกแบบ.** กรุงเทพฯ : ศูนย์หนังสือจุฬาลงกรณ์มหาวิทยาลัย.

ศุภกฤษฎิ์ นิวัฒนากูล. (2547). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

Connolly, T. and Begg, C. (2004). **Database Systems: A Practical Approach to Design, Implementation and Management, 4th ed.** Boston : Addison-Wesley.

Elmarsi, R. and Navathe, S. B. (2006). **Fundamentals of Database Systems, 5th ed.** Boston : Addison-Wesley.

Ramakrishnan, R. and Gehrke, J. (2003). **Database Management Systems, 3rd ed**. Singapore : McGraw-Hill.